

Appendix A Java Summary

Java Keywords

Primitive Types

Arithmetic Operators

Relational Operators

Logical Operators

Operator Precedence

Wrapper Classes

Syntax Templates

Variable declaration without initialization:

```
<type> <name>, <name>, <name>, . . . , <name>;
```

Variable declaration with initialization:

```
<type> <name> = <expression>;
```

Assignment:

```
<variable> = <expression>;
```

Constant declaration:

```
public static final <type> <name> = <expression>;
```

Static method definition:

```
public static <type> <name>(<type> <name>, . . . , <type> <name>)
{
    <statement>;
    ...
    <statement>;
}
```

Call on static method:

```
<method name>(<expression>, <expression>, . . . , <expression>)
```

Call on instance method:

```
<variable>.<method>(<expression>, <expression>, . . . ,
<expression>)
```

Class definition:

```
public class <class name> {
    // fields
    private <type> <name>;
    private <type> <name>;
    ...
}
```

```
// constructors

public <class name>(<type> <name>, . . . , <type> <name>) {

    <statement>;

    ...

    <statement>;

}

...

// methods

public <type> <name>(<type> <name>, . . . , <type> <name>) {

    <statement>;

    ...

    <statement>;

}

...

}
```

Constructor calling another constructor:

```
this(<expression>, <expression>, . . . , <expression>;
```

Instance method calling superclass method:

```
super.<method>(<expression>, <expression>, . . . , <expression>)
```

Constructor calling superclass constructor:

```
super(<expression>, <expression>, . . . , <expression>);
```

Specifying an inheritance relationship:

```
public class <subclass name> extends <superclass name> {  
    ...  
}
```

Implementing an interface:

```
public class <name> implements <interface> {  
    ...  
}
```

Specifying inheritance relationship and implementing interfaces:

```
public class <name> extends <superclass name>  
    implements <interface name>, <interface name>, . . . ,  
<interface name> {  
    ...  
}
```

Interface definition:

```
public interface <name> {  
    public <type> <name>(<type> <name>, . . . , <type> <name>);  
    ...  
    public <type> <name>(<type> <name>, . . . , <type> <name>);  
}
```

Abstract class:

```
public abstract class <name> {  
    ...  
}
```

Abstract method:

```
public abstract <type> <name>(<type> <name>, . . . , <type>  
<name>);
```

`return` statement:

```
return <expression>;
```


`throw` **statement:**

```
throw <exception>;
```

`assert` **statement:**

```
assert <boolean test>;
```

Array declaration:

```
<element type>[] <name> = new <element type>[<size>;
```

Array initialization:

```
<element type>[] <name> = {<value>, <value>, ..., <value>;
```

Simple `if`:

```
if (<test>) {  
    <statement>;  
    ...  
}
```

```
<statement>;  
}
```

if/else:

```
if (<test>) {  
    <statement>;  
    ...  
    <statement>;  
} else {  
    <statement>;  
    ...  
    <statement>;  
}
```

Nested if/else ending in test:

```
if (<test1>) {  
    <statement1>;  
} else if (<test2>) {  
    <statement2>;  
} else if (<test3>) {  
    <statement3>;  
}
```

Nested `if/else` ending in `else:`

```
if (<test1>) {  
    <statement1>;  
} else if (<test2>) {  
    <statement2>;  
} else {  
    <statement3>;  
}
```

`for` loop:

```
for (<initialization>; <continuation test>; <update>) {  
    <statement>;  
    ...  
    <statement>;  
}
```

For-each loop:

```
for (<type> <name> : <array or collection>) {  
    <statement>;  
    ...  
    <statement>;  
}
```

`while` **loop:**

```
while (<test>) {  
    <statement>;  
    ...  
    <statement>;  
}
```

`do/while` **loop:**

```
do {  
    <statement>;  
    ...  
    <statement>;  
} while (<test>;
```

`try/catch` **statement:**

```
try {  
    <statement>;  
    ...  
    <statement>;  
} catch (<type> <name>) {
```

```
<statement>;
```

```
...
```

```
<statement>;
```

```
}
```

Useful Methods of `ArrayList` Objects

Useful Methods of the Character Class

Useful Methods of the Collection Interface

Useful Methods of the Collections Class

Useful Methods of `DrawingPanel` Objects

Useful Methods of `File` Objects

Useful Methods of `Graphics` Objects

Useful Methods of `Iterator` Objects

Useful Methods of `Map` Objects

Constants and Useful Methods of the `Math` Class

Useful Methods of the `Object` Class

Useful Methods of `Point` Objects

Useful Methods of Random Objects

Useful Methods of Scanner Objects

Useful Methods of `String` Objects

Appendix B The Java API Specification and Javadoc Comments

The Java API Specification

Java's Application Programming Interface (API) Specification is a set of web pages that describe the classes, interfaces, and methods of the Java class libraries. You can use these pages to learn class and method names or to find details about a particular method. The API pages can be thought of as a contract between the authors of the class libraries and you, the client of those classes.

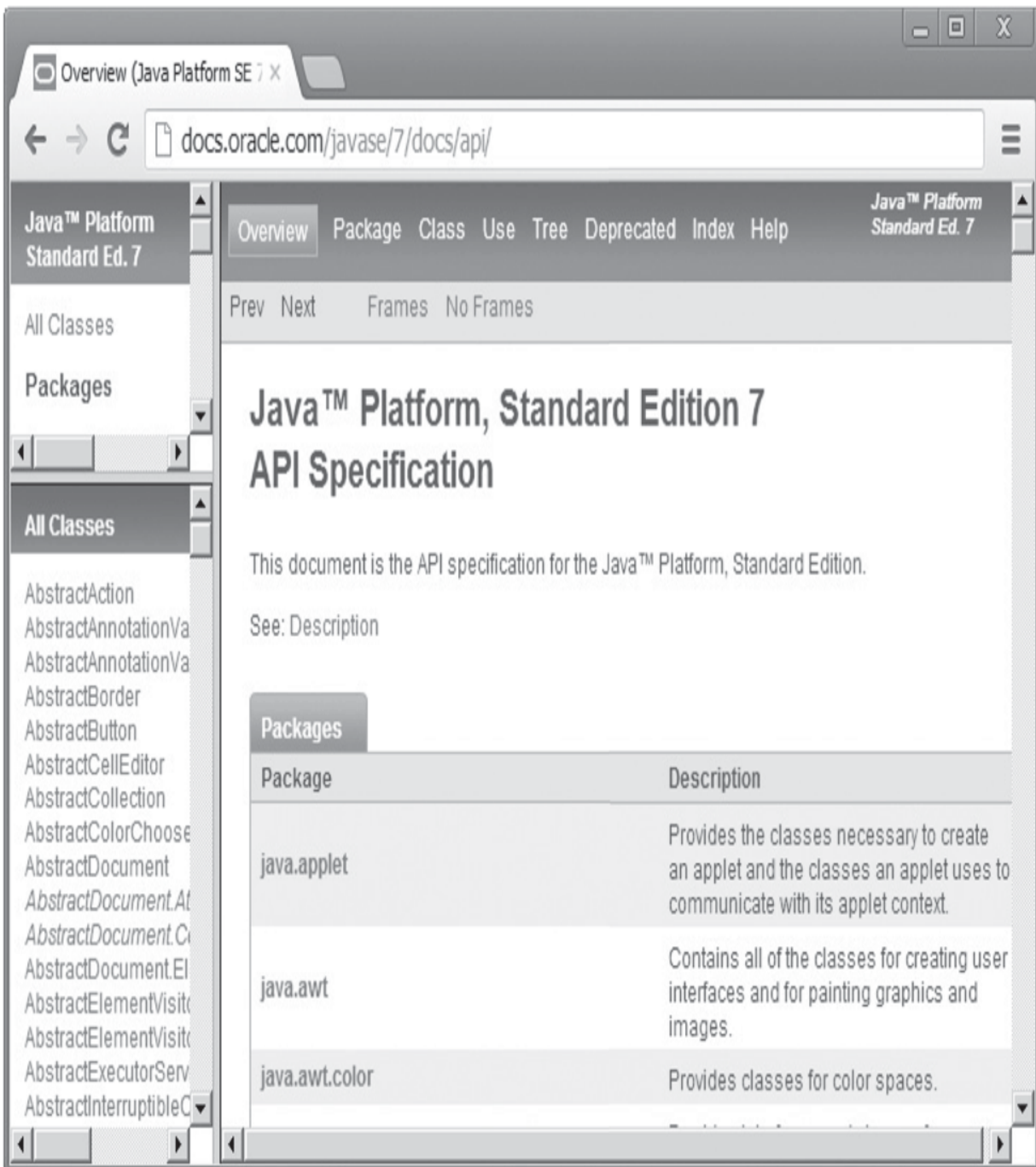
The API pages exemplify the idea of the public view of a class versus its private implementation. Each class has a set of constructors, constants, and methods that its clients can access and use. The class also has private fields, methods, and method bodies that are used to actually implement the behavior specified in the public interface. The main benefit of this separation is that you don't need to know the private implementation of the class libraries to be able to use them.

As of this writing, the current Java API Specification pages can be accessed from the following URL:

<http://docs.oracle.com/javase/10/docs/api/>

(Each new version of Java has a new specification, so this URL will change as new versions are released.) When you visit the API pages you'll see a screen that looks like **Figure B.1** .

Figure B.1 The Java API Specification



The main frame of the page (on the right) shows information about classes that you can select using the frames on the left. The lower-left frame lists all of the classes and interfaces, and the upper-left frame lists all of the packages in the class libraries. Recall that

packages are groups of related classes that you can use in your program by importing them. If you're looking for a class from a particular package, you can click that package's name in the top-left frame to filter the results in the bottom-left frame.

Once you click a class name in the bottom-left frame, information about that class appears in the main frame. At the start of the page you'll see a tree showing the names of any superclasses and any interfaces that it implements, the class's header, and a summary description of the class. The summary description gives general information about the purpose and usage of the class and may link to other relevant documentation or tutorials.

An ordered list of the contents of the class follows the summary. Listed first will be any public fields and class constants. Next will be the class's constructors, followed by its methods. Each field, method, and constructor has a line showing information such as its name, parameters, and return type, followed by a one-sentence description. You can click the name of the item to see more details about it.

For example, the following is the summary information for the

`Scanner` class's constructor to read input from a file: 

Constructor Summary

`Scanner` [\(File source\)](#)

Constructs a new `Scanner` that produces values scanned from the specified file.

Clicking on a method can provide useful details about how to use it, including a longer description, a detailed explanation of each parameter that the method requires, a description of what kind of value (if any) the method returns, and a listing of any exceptions the method may throw. These detail views allow you to learn the preconditions and postconditions of the method and to see examples of calls and their results.

For example, the following are the details about the `substring` method of the `String` class:

```
substring  
public String substring(int beginIndex,  
                        int endIndex)
```

Returns a new string that is a substring of this string. The substring begins at the specified `beginIndex` and extends to the character at index `endIndex - 1`. Thus the length of the substring is `endIndex - beginIndex`.

Examples:

```
"hamburger".substring(4, 8) returns "urge"
```

```
"smiles".substring(1, 5) returns "mile"
```

Parameters

`beginIndex`—the beginning index, inclusive

`endIndex`—the ending index, exclusive

Returns

the specified substring

Throws

`IndexOutOfBoundsException`—if the `beginIndex` is negative, if `endIndex` is larger than the length of this `String` object, or if `beginIndex` is larger than `endIndex`

Some of the methods in the API specs are marked as *deprecated*.

Deprecated

Discouraged from use.

Deprecated items are ones that Java's designers want to discourage you from using. Generally these are methods that Oracle has either renamed or decided shouldn't be called because they didn't behave correctly or safely. You might think that they should just remove these deprecated methods from Java, but doing so would break any older Java programs that call these methods. An example of a deprecated method is the `inside` method in the `Rectangle` class of the `java.awt` package:

```
inside (int x, int y)
```

Deprecated. As of *JDK version 1.1*, replaced by `contains(int, int)`.

Writing Javadoc Comments

If you write your comments in a special style called *Javadoc*, they can be used to automatically produce web pages like those in the API Specification. In fact, Oracle generates the API Specification by writing Javadoc comments in each class of the class libraries. The syntax to signify a Javadoc comment is to begin a multiline comment with `/**` rather than the usual `/*`:

```
/**  
 * This is a Javadoc comment.  
 */
```

Javadoc comments may be added to class headers, methods, class constants, public fields, and any other visible members of a class.

Many Javadoc comments also specify additional information using special syntax called tags. A *tag* is an indicator for specific information such as a description of a parameter or return value, the author of a class, the version of a file, and so on. The information in a tag can include boundary conditions, acceptable parameter ranges, and examples.

The syntax for tags is to write an `@` sign and a tag name, followed by any additional information. Several common Javadoc tags are described in the following table:

The `@author` and `@version` tags are often used on class headers. For example, the following comment can precede the header for a `Point` class:

```
/**
 * A Point object represents an ordered pair of
 * (x, y) coordinates in the 2D Cartesian plane.
 *
 * @author Marty Stepp (stepp@example.com)
 * @version 1.2 (January 12, 2018)
 */
public class Point {
    ...
}
```

The `@param` and `@throws` tags are often used on methods and constructors. Non `void` methods may also use the `@return` tag. It is best to use the three preceding tags if they supply valuable additional information that cannot be discerned from the method's header or overall comment header.

For example, the following comment can precede the header for the `distance` method of the `Point` class:

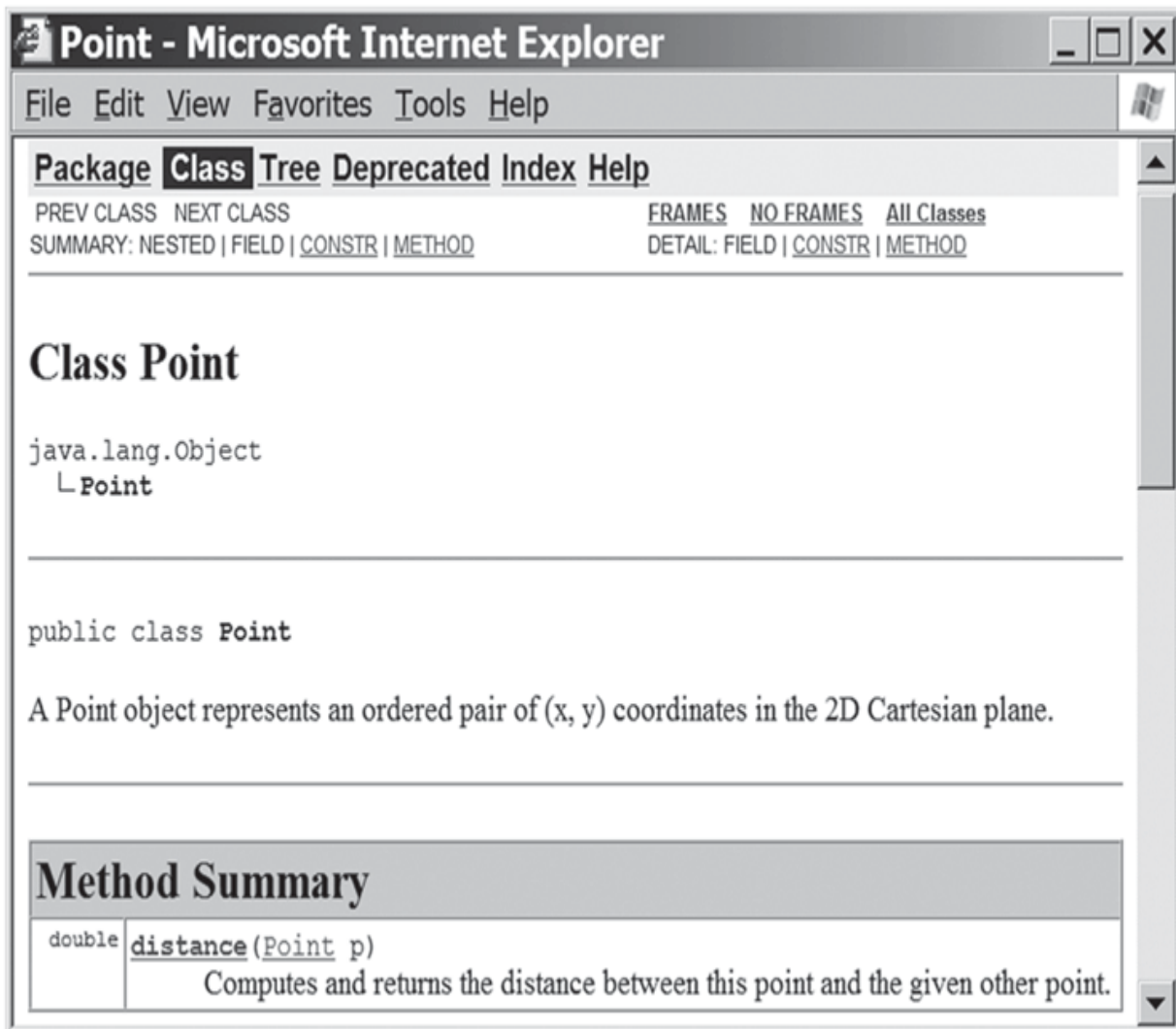
```
/**
 * Computes and returns the distance between this point
 * and the given other point.
 *
 * @param p the point to which the distance is computed
 * @return the distance, computed as the square root of
 *         the sums of the squares of the differences
 *         between the two points' x-coordinates (dx)
 *         and between their y-coordinates (dy)
 * @throws NullPointerException if p is null
 */
public double distance(Point p) {
    int dx = x - p.x;
    int dy = y - p.y;
    return Math.sqrt(dx * dx + dy * dy);
}
```

As we mentioned previously, Javadoc comments can be converted into web documentation pages. Some Java editing environments include this functionality; check your editor to see whether it is provided. If not, the JDK includes a command-line tool for generating the pages. To use it, open a terminal window to the directory of your source code files and type the following command:

```
javadoc *.java
```

The web page that is generated for the preceding `Point` class is shown in [Figure B.2](#).

Figure B.2 Generated Javadoc pages for `Point` class



Clicking on the name of a method brings up the method's detail page, as in the API Specification. The web page that is generated for the preceding `distance` method looks like this: 

Method Detail

distance

```
public double distance(Point p)
```

Computes and returns the distance between this point and the given other point.

Parameters:

`p` — the point to which the distance is computed

Returns:

the distance, computed as the square root of the sums of the squares of the differences between the two points' x-coordinates (`dx`) and between their y-coordinates (`dy`)

Throws:

`java.lang.NullPointerException` — if `p` is null

Because Javadoc comments are converted into HTML, they can contain HTML tags that will show up in any generated Javadoc

pages. You may use these tags to format your comments.

Javadoc comments provide documentation to clients that lets them know how to use your class without needing to read its source code. However, they are lengthy and can take time to write. Writing Javadoc comments is most useful when you know that your class will be used by many clients.

Oracle maintains a web site with much more information about writing Javadoc comments at the following URL:

<http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html>

Appendix C Additional Java Syntax

This appendix briefly covers several features of Java that are not otherwise shown in the textbook. It is not intended to be a complete reference; see the Java Language Specification and Java Tutorial online for more detailed coverage of these language features.

Primitive Types: `byte`, `short`, `long`, `float`

In this textbook, we have focused our attention on four primitive data types: `int`, `double`, `char`, and `boolean`. Java has four additional primitive types that are used in certain special situations. Three of these additional types (`byte`, `short`, and `long`) are variants of `int` that use a different amount of memory, providing a tradeoff between memory consumption and the range of numbers that can be represented. The fourth, `float`, is a variant of `double` that uses half the memory. It can be useful in certain applications where reduced memory usage and faster computation is more important than extremely high numeric accuracy, such as in computer games.

Many of the operators and much of the syntax that you have used with `int` and `double` work with these types. Two of these types use a suffix letter at the end of their literal values: `F` for float, and `L` for `long`. The following code declares a variable of each type:

```
byte var1 = 63;  
short var2 = -16000;
```

```
long var3 = 1234567890123456L;
```

```
float var4 = 1.2345F;
```

Ternary Operator ? :

Java has a *ternary operator* that allows you to choose between two expressions based on the value of a `boolean` test. (“Ternary” means “having three sections.”) Think of it as an abbreviated form of an `if/else` statement, except that an `if/else` chooses between two blocks of statements, while a ternary expression chooses between two expressions or values:

```
<test> ? <expression1> : <expression2>
```

A ternary expression is most useful when you want to assign a variable one of two values, or when you want to pass one of two values as a parameter or return value. For example:

```
// if d > 10.0, set x to 5; else set x to 2
double d = ...;
int x = d > 10.0 ? 5 : 2;

// e.g. "I have 3 buddies" or "I have 1 buddy"
int pals = ...;
String msg = "I have " + pals + " " + (pals == 1 ? "buddy" :
                                     "buddies");
```

```
// Returns the larger of a and b  
public static int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

Exiting a Loop: `break` and `continue`

Java has a statement called `break` that will immediately exit a `while`, `do/while`, or `for` loop. One common usage of `break` is to write a loop that performs its exit test in the middle of each iteration rather than at the start or end. The common template is to form what appears to be an infinite loop:

```
while (true) {  
    <statement>;  
    ...  
    if (<test>) {  
        break;  
    }  
    <statement>;  
    ...  
}
```

Because the `boolean` literal `true` always evaluates to `true`, this loop appears to execute indefinitely. But a test occurs in the middle of the loop. This technique is useful for solving fencepost and sentinel

problems. For example, [Chapter 5](#)  examines a sentinel problem for summing numbers until -1 . This problem can be elegantly solved with `break` as follows:

```
Scanner console = new Scanner(System.in);  
int sum = 0;  
while (true) {  
    System.out.print("next integer (-1 to quit)?");  
    int number = console.nextInt();  
    if (number == -1) {  
        break;  
    }  
    sum += number;  
}  
System.out.println("sum = " + sum);
```

The `continue` statement immediately ends the current iteration of a loop. The code execution will return to the loop's header, which will perform the loop's test again (along with the update step, if it is a `for` loop). The following loop uses `continue` to avoid negative integers:

```
for (int i = 0; i < 100; i++) {  
    System.out.print("type a nonnegative integer:");  
    int number = console.nextInt();
```



```
    if (number < 0) {  
        continue; // skip this number  
    }  
    sum += number;  
    ...  
}
```

Many programmers discourage use of the `break` and `continue` statements because these statements cause the code execution to jump from one place to another, which some people find nonintuitive. Any solution that uses `break` or `continue` can be rewritten to work without them.

It is possible to label statements and to `break` or `continue` executing from a particular label. This can be useful for immediately exiting to particular levels from a set of nested loops. This syntax is outside the scope of this textbook.

The `switch` Statement

The `switch` statement is a control structure that is similar to the `if/else` statement. It chooses one of many paths (“cases”) to execute on the basis of the value of a given variable or expression. It uses the following syntax:

```
switch (<expression>) {  
    case <value>:  
        <statements>;  
        break;  
    case <value>:  
        <statements>;  
        break;  
    ...  
    default: // optional  
        <statements>;  
        break;  
}
```

The `<expression>` used in a `switch` statement must be an integral type (`byte`, `short`, `char`, or `int`) or an enumerated type (`enum`,

discussed later in this [appendix](#)). In Java version 7 and up, you may also switch on a `String` value.

The benefit of the `switch` syntax is that you do not need to repeat the `else if` syntax or the variable's name for each path; you simply write `case` plus the next value to test. If the value does not match any of the cases, none of them is executed. The following code prints a runner's medal; if the runner was not in first through third place, no message is printed.

```
Scanner console = new Scanner(System.in);
System.out.print("In what place did you finish the race? ");
int place = console.nextInt();
switch (place) {
    case 1:
        System.out.println("You won the gold medal!!!");
        break;
    case 2:
        System.out.println("You earned a silver medal!");
        break;
    case 3:
        System.out.println("You got a bronze medal.");
        break;
}
```

The optional `default` case holds code to execute if the expression's value does not match any of the other cases, as shown in the code that follows.

The tricky part of `switch` is remembering to write a `break` statement at the end of each case. If you omit the `break` at the end of a case, the code “falls through” into the next case and also executes its code. A programmer may cause this to happen intentionally, as in the following code, but often it is done by accident and leads to bugs:

```
switch (place) {  
    case 1: // give the same response for values 1-3  
        case 2:  
        case 3:  
            System.out.println("You won a medal!!!");  
            break;  
        default:  
            System.out.println("You did not win a medal. Sorry.");  
            break;  
}
```

Many programmers avoid the `switch` statement because it is so easy to produce a bug such as that just described. In older programming languages the `switch` statement was a more efficient way to execute the `if/else` statement, but this benefit is not noticeable in Java.

The `try/ catch` Statement

The `try/catch` statement “tries” to execute a given block of code (called the “`try` block”). The statement also specifies a second “`catch` block” of code that should be executed if any code in the “`try` block” generates an exception of a particular type. It uses the following syntax:

```
try {  
    <statements>;  
} catch (<type> <name>) {  
    <statements>;  
}
```

For example, the following code attempts to read an input file and prints an error message if the operation fails:

```
try {  
    Scanner input = new Scanner(new File("input.txt"));  
    while (input.hasNextLine()) {  
        System.out.println(input.nextLine());  
    }  
}
```

```
    }  
    } catch (FileNotFoundException e) {  
        System.out.println("Error reading file: " + e);  
    }  
}
```

If you wrap all potentially unsafe operations in a method with the `try/catch` syntax, you do not need to use a `throws` clause on that method's header. For example, you do not need to declare that your `main` method throws a `FileNotFoundException` if you handle it yourself using a `try/catch` block.

Some variations of the `try/catch` syntax are not shown here. It is possible to have multiple `catch` blocks for the same `try` block, to handle multiple kinds of exceptions. It is also possible to add another block called a “`finally` block” that contains code to execute in all cases, whether an error occurs or not. This technique can be useful to consolidate cleanup code that should be run after the `try` block finishes. These syntax variations are outside the scope of this textbook.

The `assert` Statement

In [Chapters 4](#) and [5](#), we discussed preconditions, postconditions, and logical assertions. Sometimes programmers want to test logical assertions (Boolean expressions) as sanity checks in their own code. As the code runs, it will check each assertion that it reaches. If the assertion's expression is not true, the program will halt with an error.

Java supports the testing of assertions with its `assert` statement, which uses the following syntax:

```
assert <boolean test>;
```

For example, to test that a variable `x` is nonnegative, you could write the following line of code:

```
assert x >= 0;
```

In general, we expect these assertions to succeed. When an assertion fails, it signals a problem. It means that the program has a

logic error that is preventing the assumptions from holding true.

Testing of assertions can be expensive, so Java lets you control whether this feature is enabled or disabled. You can enable assertion checking in your Java editor while you are developing and testing a program to make sure it works properly. Then you can disable it when you're fairly confident that the program works and you want to speed it up. By default, assertion checking is disabled.

Enumerations: `enum`

Since [Chapter 2](#) we have seen the usefulness of class constants. Sometimes we want to create a type that has only a small number of predefined constant values. For example, suppose we are writing a program for a card game. Each card has a suit: Clubs, Diamonds, Hearts, or Spades. We could represent these values as integers (0 through 3) or strings, but these are clumsy solutions because the range of integers and strings is large, so it may be possible to slip in an invalid suit value.

In such situations, it can be useful to create an *enumerated type*, which is a simple type that has only a small number of constant values.

```
public enum <name> {  
    <name>, <name>, ..., <name>  
}
```

For example, to create an enumerated type for card suits, you could write the following code in a file named `Suit.java`:

```
public enum Suit {  
    CLUBS, DIAMONDS, HEARTS, SPADES  
}
```

The following client code uses the enumerated type:

```
// 9 of Diamonds  
int myCardRank = 9;  
Suit myCardSuit = Suit.DIAMONDS;
```

It would also be possible to create a `Card` class that has a field for the card's rank and a field of type `Suit` for the card's suit. This way an invalid suit can never be passed; it must be one of the four constants declared. You can test whether a variable of an enumerated type stores a particular value with the `==` and `!=` operators:

```
if (myCardSuit == Suit.CLUBS) {  
    // then this card is a Club  
    ...  
}
```

The `enum` concept is borrowed from past programming languages such as C, in which enumerated constants were actually represented as 0-based integers on the basis of the order in which they were declared. If you want to get an integer equivalent value for a Java `enum` value, call its `ordinal` method. For example, the call of `myCardSuit.ordinal()` returns 1 because `DIAMONDS` is the second constant declared. Every `enum` type also has a `values` method that returns all possible values in the enumeration as an array. For example, the call of `Suit.values()` returns the following array:

```
{Suit.CLUBS, Suit.DIAMONDS, Suit.HEARTS, Suit.SPADES}
```

Packages

Since [Chapter 3](#) we have used the `import` statement to make use of classes from the Java class libraries. The classes in the libraries are organized into groups called *packages*. Packages are valuable when you are working with large numbers of classes. They give Java multiple *namespaces*, meaning that there may be two classes with the same name so long as they are in different packages. Packages also help physically separate your code files into different directories, which makes it easier to organize a large Java project.

Every Java class is part of a package. If the class does not specify which package it belongs to, it is part of a nameless default package. To specify that the class belongs to some other package, place a package declaration statement as the first statement at the top of the file, even before any `import` statements or the class's header. A package declaration has the following syntax:

```
package <name>;
```

For example, to specify that the class `CardGame` belongs to the `homework4` package, you would write the following statement at the top of the `CardGame.java` file:

```
package homework4;

import java.io.*;
import java.util.*;

// This class represents the main card game logic.
public class CardGame {
    ...
}
```

Packages may be nested, indicated by dots. The packages in the Java class libraries are nested at least two levels deep, such as `java.util` or `java.awt.event`. For example, to specify that a graphical user interface file for Homework 4 belongs to the `gui` subpackage within the `homework4` package, you would write the following statement:

```
package homework4.gui;

...
```

Packages are reflected by the directory structure of the files in your Java project. For example, if a file claims to belong to the `homework4` package, it must be placed in the `homework4/` directory relative to the

root directory of your project. A file in the `homework.gui` package must be in the `homework4/gui` directory relative to the project's root.

If you write classes that are part of different packages and a class from one package wants to use a class from another, you must use an import statement for the compiler to find the classes. Remember that packages are not nested, so importing the `homework4` package does not automatically import `homework4.gui` and vice versa.

```
package general;  
  
import homework4.*;  
import homework4.gui.*;  
...
```

Packages are generally not necessary for small projects, though some editors add them to the top of all files automatically. Users of basic Java IDEs or text editors often avoid packages because they can make it harder to compile the entire project successfully. More sophisticated Java editors such as Eclipse or NetBeans handle packages better and usually provide a one-click button for compiling and executing all of the packages of a project.

Protected and Default Access

In [Chapter 8](#)'s discussion of encapsulation we learned about two kinds of access: `private` (visible only to one class) and `public` (visible to all classes). In general, you should follow the convention shown in this book of declaring all fields `private` and most methods `public` (other than internal helper methods). But Java has two other levels of access:

- `protected` access:

Visible to this class and all of its subclasses, as well as to all classes in the same package as this class.

- default (package) access:

Visible to all classes in the same package as this class.

Some programmers prefer to declare their fields as `protected` when writing classes that are part of an inheritance hierarchy. This declaration provides a relaxed level of encapsulation, because subclasses can directly access protected fields rather than having to go through accessor or mutator methods. For example, if a subclass `DividendStock` chose to extend the following `Stock` class, it would be able to directly get or set its `symbol` or `shares` field values:

```
public class Stock {  
    protected String symbol;  
    protected int shares;  
    ...  
}
```

The downside is that any class in the same package can also access the fields, which is generally discouraged. A compromise recommended by Joshua Bloch, author of *Effective Java*, is to give subclasses access to private data through protected methods instead of protected fields. This gives the class author the freedom to change the implementation of the class later if necessary.

Default access is given to a field when it has no access modifier in front of its declaration, such as in the initial versions of our `Point` class in [Chapter 8](#)  (prior to properly encapsulating it). A field or method with default access can be directly accessed by any class in the same package. We generally discourage the use of default access in most situations, since it can needlessly violate the encapsulation of the object.

Index

A

- `abs` method, [161](#), [270](#), [1140–1141](#)
- abstract classes, [647–652](#)
 - defined, [648](#)
 - interfaces vs., [647–652](#)
 - `ShareAsset` class, [648–650](#)
 - syntax for declaring, [648](#)
 - uses, [651](#)
- abstract data types. See [ADTs \(abstract data types\)](#)
- abstraction
 - defined, [560](#), [561](#)
 - representation, [574](#)
- abstract methods, [634](#), [648](#)
- Abstract Window Toolkit (AWT), [202](#)
- access
 - default, [1190](#)
 - inherited fields, [603–605](#)
 - private, [561–567](#), [604](#)
 - protected, [1190](#)
- accessors, [548–549](#), [1122](#)
- `ActionListener` interface, [632](#)
- actual parameters, [145](#), [149–150](#)
- addition (+) operator, [84–86](#)
- additive operators, [73](#), [74](#), [88](#), [247](#)

- `add` method, [671](#), [747](#)
- `addSorted` method, [1002–1008](#)
- ADTs (abstract data types), [731–733](#), [892](#)
 - defined, [731](#)
 - interfaces and, [731](#)
 - map, [746](#)
- algorithmic thinking, [3](#)
- algorithms
 - analysis, [852](#), [854](#)
 - array-traversal, [470–484](#)
 - binary search, [862–865](#)
 - concept of, [2–3](#)
 - constant-time, [858](#)
 - cubic, [860](#)
 - cumulative, [259–270](#)
 - cumulative sum, [259–261](#)
 - cumulative text, [272–274](#)
 - defined, [2](#)
 - Euclid's, [785](#)
 - exponential, [860](#)
 - fencepost, [333–339](#)
 - linear, [859](#), [861](#)
 - logarithmic, [858](#)
 - log-linear, [859](#)
 - merge sort, [873](#)
 - nested loops, [495–497](#)
 - pseudocode, [108–110](#)
 - quadratic, [859](#)

- range, [855–858](#)
- searching and sorting, [861–872](#)
- selection sort, [869–872](#)
- sequential, [447](#)
- string traversal, [481–482](#)
- AND (&&) operator, [258](#), [288](#)
 - precedence, [343](#)
 - short-circuited evaluation, [346](#)
 - truth table for, [341](#)
- annotation, [964](#)
- API (Application Programming Interface) Specification, [8](#)
- Apple Computer, [538](#)
- Application Programming Interface (API) Specification, [1176–1178](#)
- applications, [4](#)
- arguments. See [parameters](#)
- arithmetic notations, [819](#)
- arithmetic operators, [69–72](#), [1161](#)
- `ArrayIndexOutOfBoundsException` Exception, [455](#), [472](#), [491](#), [950](#)
- `ArrayIntList` class, [932](#), [941](#), [990](#), [1008](#), [1084](#)
 - adding an iterator, [957–963](#)
 - advanced features, [955–963](#)
 - constructor and constant, [944–945](#)
 - convenience methods, [952–955](#)
 - converting into `ArrayList<E>`, [963–968](#)
 - encapsulation, [938–939](#)
 - garbage collector, [965](#)
 - inner class, [966–968](#)

- preconditions and postconditions, [945–948](#)
- printing/adding, [932–937](#)
- resizing, [955–957](#)
- throwing exceptions, [949–955](#)
- `ArrayList`, [667–721](#), [723](#), [841](#)
 - adding to, [679–683](#)
 - adding values to, [671–672](#)
 - benefits of, [730](#)
 - `Collections.sort` method, [689](#)
 - complete program, [677–679](#)
 - defined, [667](#), [668](#)
 - for-each loop, [684–686](#)
 - `LinkedList` vs., [724–726](#)
 - `for` loop and, [680–683](#)
 - methods, [675](#)
 - natural ordering of values, [691–694](#)
 - operations, [669–674](#)
 - printing, [670](#)
 - removing from, [679–683](#)
 - searching methods, [674–677](#)
 - syntax for constructing, [669](#)
 - wrapper classes, [686–688](#)
- `ArrayList` objects, methods of, [1168](#)
- arrays, [447–534](#), [1140–1141](#)
 - accessing, [452–455](#)
 - advanced techniques, [488–497](#)
 - auto-initialization, [450](#)
 - basics of, [448–470](#)

- buffer overruns, [457–458](#)
- case study, [509–519](#)
- command-line arguments, [494–495](#)
- constructing, [448–452](#)
- declaring, [449](#)
- defined, [447](#), [448](#)
- equality, testing for, [475–477](#)
- filled, [933](#)
- for-each loop, [467–468](#)
- heap implementation, [1106–1108](#)
- index, [448](#)
- initializing, [455–456](#)
- jagged, [499–503](#)
- loops, [467–468](#)
- merging, [873–876](#)
- and methods, [464–466](#)
- multidimensional, [497–503](#)
- nested loop algorithms, [495–497](#)
- of objects, [493–494](#)
- of pixels, [504–509](#)
- printing, [471–473](#)
- program, [456–461](#)
- random access, [461–462](#)
- rectangular two-dimensional, [497–499](#)
- reversing, [477–481](#)
- searching and replacing, [473–475](#)
- sequential search, [861–862](#)
- shifting values in, [488–493](#)

- sorted, [1084](#)
- splitting, [873–876](#)
- traversal algorithms, [470–484](#)
- traversing, [448–452](#)
- unfilled, [933](#)
- unsorted, [1084](#)
- `Arrays.asList` method, [1141–1142](#)
- `Arrays` class, [468–469](#)
 - `copyOf` method, [469](#)
 - `copyOfRange` method, [469](#)
 - `deepToString` method, [499](#)
 - `equals` method, [469](#), [476–477](#)
 - `parallelSort` method, [845](#)
 - `sort` method, [845](#)
 - `toString` method, [469–470](#), [499](#)
 - `Arrays.stream` method, [1140–1141](#)
- ASCII, [275](#)
- assertions, [358–366](#)
 - defined, [358](#)
 - example, [362–366](#)
 - provable, [360](#)
 - reasoning about, [360–362](#)
- `assert` statement, [1187](#)
- assignment operators, [83](#), [88](#), [247](#)
- assignment statements, [78](#), [79–80](#), [81–82](#), [83](#)
- asterisk (*), [65](#), [69](#)
- auto-initialization, [450](#)

- automatic garbage collection, [981](#)
- average case analysis, [861](#)

B

- backslash character (\), [16](#), [69](#)
- back trace, [170](#)
- backtracking, recursive, [801–818](#)
- backward compatibility, [629](#)
- base case, [770–771](#), [780](#), [782–785](#), [799](#), [804](#)
- behavior, object, [536](#), [540](#)
- Benford’s Law case study, [509–519](#)
- best case analysis, [862](#)
- big-Oh notation, [858](#)
- binary files, [395](#)
- binary heaps, [1102](#)
- binary numbers, [4–5](#)
- binary search, [842–845](#), [862–865](#)
 - defined, [842](#)
 - details, [867–868](#)
 - implementing, [862–865](#)
 - recursive, [865–867](#)
 - runtime of, [865](#)
- `binarySearch` method, [842](#), [867–868](#)
- binary search tree, [740](#), [1052–1067](#)
 - building, [1055–1058](#)
 - complexity, [1066–1067](#)
 - defined, [1052](#), [1054](#)
 - property, [1053–1055](#)

- searching, [1062–1066](#)
- binary tree, [1028–1071](#)
 - branch nodes, [1029](#), [1030](#)
 - counting leaves, [1051–1052](#)
 - counting levels, [1049–1050](#)
 - defined, [1029](#)
 - degenerate, [1067](#)
 - height of, [1051](#)
 - kinds of, [1031](#)
 - leaf nodes, [1030–1031](#)
 - operations of, [1048–1052](#)
 - parent/child/sibling/ancestor/descendant, [1031](#)
 - root of, [1029](#), [1031–1032](#)
 - sum of, [1048–1049](#)
 - traversals, [1033–1048](#)
- block copy operation, [956](#)
- `Board` object, [809–810](#)
- Body Mass Index (BMI), [76](#)
 - case study, [290–305](#)
- boilerplate code, [427](#)
- Boole, George, [339](#)
- Boolean expression, [339–341](#), [353–354](#)
- `boolean` flag, [908](#)
- `boolean` type, [64](#), [69](#), [339–354](#)
 - defined, [320](#)
 - flags, [348–350](#)
 - values, [339–340](#)
 - variables, [340](#), [348–350](#)

- Boolean Zen, [350–352](#)
- bounds checking, [458](#)
- bound variables, [1137](#), [1138](#)
- boxing, [688](#), [690–691](#)
- branch nodes, [1029](#)
- `break` statement, [1183–1184](#)
- bubbling, [1103](#)
- buckets, [1087](#)
- buffer overruns, [457–458](#)
- bugs, [28](#)
 - off-by-one, [462–463](#)
 - Y2K (millennium), [564](#)
 - Zune, [345](#)
- bytecodes, [7](#)
- `byte` type, [64](#)

C

- calls, [602–603](#)
 - complex, interpretation, [622–625](#)
 - constructor, [605–607](#)
 - recursive, [783](#), [806](#), [810](#)
- call stack, [776](#)
- `canRead` method, [425](#)
- capacity, list, [934](#)
- case studies
 - Benford's Law, [509–519](#)
 - Body Mass Index (BMI), [290–305](#)
 - `DrawFigures`, [40–46](#)
 - financial class hierarchy, [639–642](#)
 - hourglass figure, [115–123](#)
 - lottery program, [743–745](#)
 - merge sort, implementation, [873–881](#)
 - `NumberGuess`, [366–375](#)
 - perfect numbers, [1146–1156](#)
 - prefix evaluator, [818–824](#)
 - projectile trajectory, [178–186](#)
 - pyramids, [224–230](#)
 - sieve algorithm, [734–737](#)
 - `Stock` class, [573–582](#)
 - vocabulary comparison, [701–714](#)

- `WordCount`, [750–753](#)
- zip code lookup, [427–434](#)
- casting, [74–76](#), [612](#)
- central processing unit (CPU), [3](#)
- chaining, [304–305](#)
- `Character` class, methods of, [1169](#)
- characters, literals, [69](#)
- `charAt` method, [168](#), [285](#), [482](#)
- `char` type
 - defined, [64](#)
 - example, [64](#), [69](#)
 - `int` vs., [271–272](#)
 - literal values, [270](#)
 - `String` vs., [271](#)
- checked exception, [398](#)
- `checkIndex` method, [953](#), [954](#)
- `Circle` class, [628](#), [636](#)
- circles, area and perimeter, [633](#)
- Class, Responsibilities, Collaborators (CRC) card, [576](#)
- classes, [535–591](#)
 - abstract, [647–652](#)
 - `ArrayIntList`. See `ArrayIntList` class
 - `Arrays`, [468–470](#)
 - case study, [573–582](#)
 - `Character`, [273](#)
 - cohesion, [576](#)
 - `Color`, [208–210](#)

- complexity, [858–860](#)
- concrete, [651](#)
- constants, [113–115](#), [119–122](#)
- defined, [10](#), [166](#)
- design(ing), [573–582](#), [640–644](#)
- `DrawingPanel`, [202–203](#)
- generic, [668–669](#)
- `Graphics2D`, [629–631](#)
- `IllegalArgumentException`, [282](#)
- inner, [966–968](#)
- `Integer`, [897–898](#)
- invariants, [567–571](#)
- `LinkedList`. See `LinkedList` class
- `LinkedList<E>`, [975](#), [1011–1019](#)
- `Math`, [158–161](#)
- `Object`, [609–610](#)
- objects. See [objects](#)
- `PrintStream`, [420–424](#)
- `Scanner`, [910](#)
- `TimeSpan`, [571–573](#)
- wrapper. See [wrapper classes](#)
- class headers, [11](#)
- client code, [537](#)
- clients, defined, [537](#)
- `Cloneable` interface, [632](#)
- clustering, [1088](#)
- code, [6](#)

- boilerplate, [427](#)
- client, [537](#)
- evaluation, [917–920](#)
- legacy, [593](#)
- reuse, [593](#)
- cohesion, [302–303](#), [576](#)
- `Collection` interface, [732](#)
 - useful methods of, [724](#), [1169](#)
- collections, [723–724](#). See also [lists](#); [maps](#); [sets](#)
 - higher-order operations on, [1139–1146](#)
 - overview, [753–754](#)
- `Collections` class, methods of, [732](#)
- `Collections.sort` method, [689](#)
- collisions, [1087–1092](#)
- `Color` class, [208–210](#)
 - constants, [209](#)
- `Color` object, [506–509](#)
 - methods, [507](#)
- colors
 - background, [208](#)
 - graphics, [208–210](#)
- command-line arguments, [494–495](#)
- comments
 - defined, [22](#)
 - forms, [22–23](#)
 - Javadoc, [23](#), [1178–1181](#)
 - pre/post, [281–282](#)
 - programming error, [29](#)

- single-line, [23](#)
- writing, [1178–1181](#)
- `Comparable` interface, [632](#), [689–701](#), [848](#)
 - implementation, [692](#), [694–701](#)
- comparators
 - custom ordering with, [848–851](#)
 - defined, [740](#)
 - implementation, [848–849](#)
 - and methods, [851](#)
- `compare` method, [848–849](#)
- `compareTo` method, [692](#), [695–697](#), [848–849](#)
- compilation errors, [24](#)
- compilers, [6–7](#)
- complex expressions, [66](#)
- complexity, [852–860](#)
 - binary search tree, [1066–1067](#)
 - classes, [858–860](#)
 - defined, [852](#)
 - empirical analysis, [855–858](#)
 - managing, [103–115](#)
 - time, [852](#)
- computers, [2–10](#)
- concatenation, string, [84–87](#)
- concrete classes, [651](#)
- conditional execution, [243–319](#)
 - case study, [290–305](#)
 - cumulative algorithms, [259–270](#)
 - `if/else` statements, [244–258](#)

- methods with, [279–290](#)
- return values and, [283–287](#)
- text processing, [270–278](#)
- conditions
 - multiple, [258](#)
 - postconditions, [279](#)
 - preconditions, [279](#)
- console
 - defined, [9](#)
 - input, [9](#), [173](#)
 - interaction, [9](#)
 - output, [9](#)
 - window, [9](#)
- constants
 - class, [113–115](#), [119–122](#)
 - declaration, [114](#)
 - `Font` class, [217](#)
 - `Math` class, [160](#)
 - parameters vs., [156](#)
- constant-time algorithm, [858](#), [865](#)
- constructors, [173](#), [552–560](#), [980](#)
 - default, [555](#)
 - headers, [553](#)
 - implementation, [578–582](#)
 - multiple, [559–560](#)
 - redeclaring fields in, [556](#)
 - syntax, [555](#)
 - using `void` keyword with, [555–556](#)

- consuming input, [406–407](#)
- `contains` method, [675](#)
- `continue` statement, [1183–1184](#)
- contract, [932](#)
- control structure, [94](#)
- control variable, [106](#)
- `copyOf` method, [469](#), [956](#)
- `copyOfRange` method, [469](#)
- counter, [266](#)
- coupling
 - defined, [303–304](#)
 - loose, [575](#)
 - minimizing, [303](#)
- C programming language, [14](#), [65](#), [458](#)
- C# programming language, [458](#)
- C++ programming language, [14](#), [458](#)
- CPU (central processing unit), [3](#)
- CRC (Class, Responsibilities, Collaborators) card, [576](#)
- cubic algorithm, [860](#)
- cumulative algorithms, [259–270](#)
 - cumulative product, [280–281](#)
 - cumulative sum, [259–261](#), [265–267](#)
 - defined, [259](#)
 - `min/max` loops, [261–265](#)
 - roundoff errors, [267–270](#)
 - text, [272–274](#)
- cumulative concatenation, [273](#)
- cumulative product, [280–281](#)

- cumulative sum, [259–261](#)
 - execution, [260–261](#)
 - with `if/else` statements, [265–267](#)
 - pseudocode, [259](#)
- cumulative text algorithms, [272–274](#)
- curly brace characters (`{}`), [11](#), [97–98](#)
- current directory, [409](#)
- cursor
 - input, [405](#)
 - output, [17](#), [18](#)

D

- data
 - availability, [393](#)
 - munging, [428](#)
 - ownership, [305](#)
 - processing, origin of, [394](#)
 - recursive functions and, [781–797](#)
- data structures, [723](#)
- data types, [64–65](#). See also specific types
- dead-end, [804](#), [811](#), [813](#)
- debugging, [24](#), [28](#)
- decimal numbers, [4](#), [5](#)
- decision tree, [802–803](#), [806](#)
- declarations
 - array, [449](#)
 - import, [178](#)
 - static method, [153](#)
- declarations, variable, [77–78](#)
 - defined, [77](#)
 - variations, [81–82](#)
- decomposition. See also [procedural decomposition](#)
- decrementing, [81](#), [87](#), [99](#)
- decrement (--) operator, [87–90](#)
- `deepToString` method, [499](#)
- default access, [1190](#)

- default constructor, [555](#)
- defensive programming, [283](#)
- definite loops, [63](#), [321](#)
- degenerate tree, [1067](#)
- deprecated items, [1178](#)
- dequeuing, [896](#)
- design
 - class, [640–644](#)
 - inheritance and, [625–631](#)
 - object-oriented. See [object-oriented design \(OOD\)](#)
 - procedural, [300–305](#)
- diamond operator, [669](#)
- difference operation, [742](#)
- digital, [4](#)
- digit-matching algorithm, [369](#)
- directories, [409–412](#)
 - current, [409](#)
 - defined, [409](#)
 - recursion, [790–794](#)
 - working, [409](#)
- `DividendStock` class, [606–608](#)
- division operator (`/`), [69–70](#)
- DOS, [538](#)
- dot notation, [159](#), [544](#)
- double backslash characters (`\`), [21](#)
- double hashing, [1092](#)
- `doubleNumber` method, [151](#), [152](#)
- `double` type, [65](#)

- conversion of `int` into, [75–76](#)
- defined, [64](#)
- in scientific notation, [69](#)
- doubly linked list, [1014](#)
- `do/while` loops, [330–332](#). See also [loops](#)
 - flow of, [331](#)
 - syntax, [331](#)
- `drawBox` method, [33–36](#), [156–157](#)
- `DrawFigures` case study, [40–46](#)
 - final version, [43–44](#)
 - flow of execution, [44–46](#)
 - structured version, [41–42](#)
- `DrawingPanel` objects, [202–203](#), [218](#)
 - case study, [224–230](#)
 - methods, [202](#), [504](#), [1172](#)
 - size, [211](#)
- `drawLine` method, [203](#)
- `drawOval` method, [207](#), [208](#)
- `drawRect` method, [206](#), [208](#), [215](#)
- `drawString` method, [215](#), [220](#)
- `drawText` method, [220](#)
- `drawTriangle` method, [774–776](#)
- dummy node, [1013](#)
- dummy values, [327](#)

E

- effect-free programming, [1120–1122](#)
- elements, [723](#). See also [arrays](#)
 - auto-initialization, [450](#)
 - defined, [448](#)
- empirical analysis, [852](#)
 - complexity, [855–861](#)
- encapsulation, [560–573](#)
 - class invariants, [567–571](#)
 - defined, [560](#)
 - internal implementation changes, [571–572](#)
 - `Point` class, [611](#)
 - poor, [564](#)
 - private access, [561–567](#)
 - of superclass, [604](#)
- enqueueing, [896](#)
- enumerated types (enum), [1185–1186](#)
- epoch, [859](#)
- equality
 - array, [475–476](#)
 - object, [255–256](#)
- equality operator (`==`), [246](#), [247](#)
- `equalsIgnoreCase` method, [256](#)
- `equals` method, [256](#), [469](#), [475–477](#), [610–613](#), [868](#)

- Erle, Schuyler, [428](#)
- errors. See also [exceptions](#)
 - compilation, [24](#)
 - logic, [24](#), [28](#)
 - off-by-one, [462–463](#)
 - program, [24–28](#)
 - roundoff, [267–270](#)
 - runtime, [24](#), [39–40](#)
 - syntax, [24–28](#)
 - user, [354–358](#)
- escape sequences, [15–16](#)
- Euclid’s algorithm, [785](#)
- evaluation
 - code, [917–920](#)
 - expression, [65](#), [908–920](#)
- evaluator, [914–920](#)
- exceptions. See also [errors](#)
 - checked, [398](#)
 - defined, [170](#)
 - thrown, [170](#), [279–283](#), [949–955](#)
 - types, [950](#)
- execution
 - conditional. See [conditional execution](#)
 - flow analysis, [44–46](#)
 - `for` loops, [96](#)
 - program, [6](#)
- exponential algorithm, [860](#)
- exponentiation, integer, [781–783](#)

- expressions
 - Boolean, [339–340](#), [341](#), [353–354](#)
 - complex, [66](#)
 - defined, [65](#)
 - evaluating, [65](#)
 - operators, [66](#)
 - values, calculating, [79](#)
- `extends` keyword, [596](#)
- extensions
 - class, [596–600](#)
 - file, [8–9](#)

F

- factorials
 - computing, [280](#)
 - defined, [280](#)
 - mathematical definition of, [281](#)
- factoring
 - defined, [257](#)
 - `if/else` statements, [256–258](#)
- fencepost loop, [333–337](#). See also [loops](#)
 - defined, [333](#)
 - with `if` statement, [334–337](#)
 - sentinel, [337–338](#)
- fields, [541–543](#)
 - defined, [541](#)
 - encapsulated, [562](#)
 - inherited, accessing, [603–605](#)
 - private, [561–567](#)
 - redeclaring in constructors, [556](#)
- FIFO (First-In, First-Out), [893](#)
- figures, drawing, [40–46](#)
- `FileNotFoundException`, [398–399](#), [402](#)
- `File` objects, [396](#), [400](#), [790–791](#), [1173](#)
- files, [1145–1146](#)
 - absolute path, [409](#)

- advanced processing, [420–427](#)
- binary, [395](#)
- defined, [393](#)
- extensions, [8–9](#), [394–395](#)
- input, [412–414](#)
- line-based processing, [413–419](#)
- output, [420–425](#)
- paths, [409–410](#)
- processing, [392–446](#)
- reading, [393–400](#)
- relative path, [409](#)
- structure of, [403–407](#)
- text, [395](#)
- token-based processing, [401–413](#)
- `Files.lines` method, [1145](#)
- `fillOval` method, [208](#)
- `fillPolygon` method, [799](#)
- `fillRect` method, [208](#), [220](#)
- `filter` modifier, [1132](#)
- `final` keyword, [114](#)
- financial class hierarchy case study, [639–642](#)
- `findNextToken` method, [911](#)
- first-class citizen, [1123–1124](#)
- first-class functions, [1123–1124](#)
- flags, boolean, [348–349](#)
- floating-point numbers, [72](#)
- `float` type, [64](#), [65](#)

- flow of control, [34–36](#)
 - of `do/while` loops, [331](#)
 - of execution analysis, [44–46](#)
 - of `if/else` statement, [246](#)
 - of `if` statement, [248](#)
 - of `for` loop, [94](#)
 - of nested `if/else` statements, [250](#), [251](#)
 - of `while` loop, [321](#)
- folders, [409](#)
- `Font` class, [216](#)
 - constants, [217](#)
- fonts
 - common names, [217](#)
 - defined, [216](#)
 - graphics, [215–218](#)
- for-each loop, [791](#)
 - `ArrayList` and, [684–685](#)
 - arrays, [467–468](#), [475](#)
 - iterators, [729–730](#)
 - syntax, [467](#)
 - uses, [467](#)
- `forEach` method, [1142](#)
- `for` loop, [63](#), [92–102](#), [212](#), [767](#), [794](#), [812–813](#)
 - and `ArrayList`, [679–680](#)
 - body, [93](#)
 - as control structures, [94](#)
 - decrementing, [99](#)

- execution, [96](#), [99](#)
- flow of, [94](#)
- nested, [100](#)
- patterns, [98–99](#)
- scope, [105](#)
- syntax, [93](#)
- tracing, [94–97](#)
- formal parameter, [145](#), [149](#)
- formal verification, [360](#)
- format specifiers, [276–277](#)
- format string, [276](#)
- `Formattable` interface, [632](#)
- forward slash (/), [69–70](#)
- fractal geometry, [797](#)
- free variables, [1137](#), [1138](#)
- functional interfaces, [1129](#)
- functional programming, [1119–1156](#)
 - defined, [1120](#)
 - effect-free programming, [1120–1122](#)
 - first-class functions, [1123–1129](#)
 - function closures, [1136–1139](#)
 - higher-order function, [1139–1146](#)
 - influences on Java, [1122–1123](#)
 - perfect numbers case study, [1146–1156](#)
 - streams, [1129–1136](#)
- function closures, [1136–1139](#)
- functions, recursive, [781–797](#)
 - advantages of, [783](#)

- execution of, **782–783**

G

- garbage collection, [981](#)
- garbage collector, [965](#)
- Gauss, Carl Friedrich, [161](#)
- GB (gigabytes), [5](#), [6](#)
- GCD (greatest common divisor), [784–787](#)
- generic class, [668–669](#)
- `getBlue` method, [507](#)
- `getGraphics` method, [203](#), [629](#)
- `getGreen` method, [507](#)
- `getInput` method, [427](#)
- `get` method, [673](#), [727](#), [747](#)
- `getPixels` method, [504–505](#)
- `getRed` method, [507](#)
- gigabytes (GB), [5](#), [6](#)
- `giveProblems` method, [1124–1125](#)
- global variables, [1121](#)
- graphics
 - case study, [224–230](#)
 - colors, [208–210](#)
 - defined, [201](#)
 - `DrawingPanel` class, [202–203](#)
 - fonts, [215–218](#)

- images, [218–220](#)
- introduction to, [202–209](#)
- loops for, [211–215](#)
- procedural decomposition with, [220–224](#)
- shapes, [203–207](#)
- text, [215–218](#)
- `Graphics` class, [798–799](#)
- `Graphics2D` class, [629–631](#)
- `Graphics` objects, [204–210](#), [1171](#)
- greater than ($>$) operator, [246](#)
- greater than or equal to ($>=$) operator, [246](#)
- greatest common divisor (GCD), [784–787](#)
- growth rates, [857](#)

H

- hailstone sequences, [263](#)
- hard disk, [3](#)
- hardware, [3](#)
- has-a relationship, is-a relationship vs., [628](#)
- `hashCode` method, [1095–1097](#)
- hash codes, [738](#)
- hash function, [1087](#)
- hashing, [1084–1099](#)
 - array set implementations, [1084–1085](#)
 - collisions, [1087–1092](#)
 - double, [1092](#)
 - functions/tables, [1085–1087](#)
 - map implementation, [1098–1099](#)
 - non-integer data, [1095–1098](#)
 - rehashing, [1092–1095](#)
 - separate chaining, [1088–1092](#)
- `HashMap` interface, [746](#)
- `HashSet` class, [738–741](#)
- hash table, [738](#)
- `hasNextInt` method, [355](#), [356](#)
- `hasNextLine` method, [414](#)
- headers
 - class, [11](#)

- constructors, [553](#)
- instance methods, [544–545](#)
- method, [12](#)
- `HeapPriorityQueue`, [1111](#)
- heaps
 - adding to, [1103–1105](#)
 - array implementation, [1106–1109](#)
 - binary, [1102](#)
 - complete tree, [1102](#)
 - defined, [1102](#)
 - min/max, [1101](#)
 - priority queue, [1099–1101](#)
 - `PriorityQueue<E>` methods, [1101](#)
 - removing from, [1103–1104](#)
 - sorting algorithms, [1110–1111](#)
- Hello, world!, [10–14](#), [421–422](#)
- helper methods, [794–797](#)
- heuristics
 - defined, [300](#)
 - object-oriented design, [574–576](#)
 - procedural design, [300–305](#)
- hierarchies
 - class, [620](#)
 - inheritance, [595–596](#)
 - nonprogramming, [594–596](#)
 - shape class, [638](#)
- higher-order function, [1139–1146](#)
- hinting, [369–372](#)

- Hollerith, Herman, **394**
- Hopper, Grace, **28**
- hourglass figure case study, **115–123**
- `hypotenuse` method, **164**

- IBM, [538](#)
- identifiers, [18–19](#)
- IDEs (Integrated Development Environments), [9](#)
- IEEE (Institute of Electrical and Electronics Engineers), [280](#)
- `if/else` statements, [244–258](#), [772–773](#), [852](#), [875–876](#), [936](#)
 - cumulative sum with, [265–267](#)
 - defined, [245](#)
 - factoring, [256–257](#)
 - flow of, [246](#)
 - multiple conditions, [258](#)
 - nested, [248](#)
 - object equality, [255–256](#)
 - overview, [245](#)
 - relational operators, [246–248](#)
 - syntax, [245](#)
 - wrong construct selection, [253–255](#)
- `if` statement
 - defined, [244](#)
 - with fencepost, [334–337](#)
 - flow of, [244](#), [248](#)
 - sequential, flow of, [249](#)
 - syntax, [244](#)
- `IllegalArgumentException`, [282](#), [578](#), [949](#)

- `IllegalStateException`, **951**
- illegal values, testing for, **290**
- images, **218–220**
- immutable objects, **171**
- implicit parameter, **546–548**, **611**
- import declarations, **178**
- inchworm approach, **1007–1008**
- incrementing, **81**, **87**
- increment (**++**) operator, **87–90**
- indefinite loops, **321**
- indexes
 - arrays, **448**
 - defined, **168**
 - `String`, **286**
- `indexOf` method, **284–285**, **675**, **841**, **953**
- `IndexOutOfBoundsException`, **952**
- infinite loops, **107–108**, **320–321**
- infinite recursion, **770**, **786**, **787–790**
- infix notation, **818**, **819**
- inheritance
 - basics, **593–602**
 - code reuse, **593**
 - defined, **598**
 - and design, **625–631**
 - extending class, **596–600**
 - hierarchies, **595–596**
 - interfaces. See **interfaces**
 - misuse of, **625–627**

- nonprogramming hierarchies, [594–596](#)
- overriding methods, [600–602](#)
- polymorphism and, [615–625](#)
- single, [598](#)
- subclass, [598](#)
- superclass, [598](#), [602–615](#)
- initialization
 - arrays, [450](#), [456–457](#)
 - objects, [552–560](#)
 - parameters, [151](#)
 - variables, [95](#)
- inner class, [966–968](#)
- inorder traversals, [1034](#)
- input
 - console, [9](#), [173](#)
 - consuming, [405–406](#)
 - cursor, [405–406](#)
 - files, [412–414](#)
 - tokenizing, [175](#)
- insertion point, [868](#)
- instance methods, [543–546](#)
 - accessors, [548–549](#), [1122](#)
 - defined, [543](#)
 - headers, [544–545](#)
 - implicit parameters, [546–547](#)
 - mutators, [548–549](#), [1122](#)
 - syntax, [546](#)
- `instanceof` keyword, [613–615](#)

- instances, objects as, **166**
- `Integer` class, **897–899**
- integers exponentiation, **781–783**
- Integrated Development Environments (IDEs), **9**
- interactive programs, **173–182**
 - defined, **173**
 - sample, **176–178**
- interfaces, **631–639**
 - abstract classes vs., **647–652**
 - and abstract data types, **731**
 - `ActionListener`, **632**
 - benefits of, **637–639**
 - `Cloneable`, **632**
 - `Collection`, **724, 732**
 - `Comparable`, **632, 689–701, 848**
 - defined, **631**
 - `Formattable`, **632**
 - `HashMap`, **746**
 - implementation, **632, 634–637**
 - `IntList`, **1008–1011**
 - `List`, **731, 1141–1142**
 - `Runnable`, **632**
 - `Serializable`, **632**
 - for shapes, **632–634**
 - syntax for declaring, **634**
 - `TreeMap`, **746**
- Internet worms, **458**

- intersection operation, [742](#)
- `IntList` interface, [1008–1011](#)
- `IntStream.iterate` method, [1153](#)
- `IntStream.of` method, [1132–1133](#)
- `IntStream.range` method, [1130](#), [1131](#), [1147–1148](#)
- `int` type
 - conversion into `double`, [75–76](#)
 - defined, [64](#)
 - integer literals, [68](#)
- `intValue` method, [686](#)
- invariants, class, [567–571](#)
 - defined, [569](#)
 - enforcing, [569–570](#)
- is-a relationship, [595](#), [598](#)
 - vs. has-a relationship, [628](#)
- `isDirectory` method, [790](#)
- `isEmpty` method, [953](#), [1018](#)
- `isMultiple` method, [1136](#), [1137](#)
- iterations, [95](#), [764](#)
 - solution converted to recursion, [767–769](#)
- iterative approach, [764](#)
- iterative enhancement, [31](#)
- `Iterator` interface, [632](#)
- `Iterator` objects, methods of, [1174](#)
- iterators, [727–730](#)
 - `ArrayIntList`, [957–963](#)

- calling `next` on, [729–730](#)
- defined, [728](#)
- in for-each loops, [729–730](#)
- linked lists, [1015–1017](#)
- methods, [728](#)

J

- jagged arrays, [499–503](#)
- Java
 - bytecodes, [7](#)
 - class libraries, [8](#)
 - coding conventions, [13](#)
 - defined, [7](#)
 - need for, [7–8](#)
 - platform independence, [8](#)
 - programming environment, [8–10](#)
 - runtimes, [7](#)
- `java.awt` package, [208](#), [493](#), [539](#), [632](#)
- Java Collections Framework
 - lists, [723–737](#)
 - maps, [745–753](#)
 - sets, [737–745](#)
- Javadoc comments, [23](#), [1178–1181](#)
- `java.io` package, [632](#)
- `java.lang` package, [178](#)
- `java.nio.file`, [1145](#)
- Java Runtime Environment (JRE), [7](#)
- `java.util` package, [178](#), [632](#), [669](#), [723](#), [724](#), [842](#), [1088](#)
- Java Virtual Machine (JVM), [7](#)
- JRE (Java Runtime Environment), [7](#)

- JShell, [67–69](#)
- JVM (Java Virtual Machine), [7](#)

K

- KB (kilobytes), [5](#), [6](#)
- Kernighan, Brian, [14](#), [28](#), [103](#)
- `keySet` method, [748–749](#)
- keywords, [1161](#)
 - defined, [19](#)
 - `extends`, [598](#)
 - `final`, [114](#)
 - `instanceof`, [613–615](#)
 - `new`, [174](#), [494](#)
 - `private`, [561](#)
 - `public`, [32](#)
 - reserved, [19–20](#)
 - `static`, [33](#), [544](#)
 - `this`, [557–559](#)
 - `void`, [33](#)
- kilobytes (KB), [5](#), [6](#)
- Knuth, Don, [2](#), [3](#)

L

- lambda expression, [1126–1129](#)
- `lastIndexOf` method, [677](#)
- leaf nodes, [1030–1031](#)
 - counting, [1051–1052](#)
- legacy code, [593](#)
- `length` method, [543](#)
- less than ($<$) operator, [246](#)
- less than or equal to ($<=$) operator, [246](#)
- lexicographic ordering, [1054](#)
- libraries, defined, [8](#)
- LIFO (Last-In, First-Out), [893](#), [894](#)
- linear algorithm, [859](#), [861](#)
- linear probing, [1087](#), [1088](#)
- line-based file processing, [413–419](#)
 - defined, [413](#)
 - token-based combination, [415–419](#)
- lines, drawing, [203–208](#)
- `LinkedList` class, [1008](#)
 - `addSorted` method, [1002–1008](#)
 - appending `add`, [991–995](#)
 - inchworm approach, [1007–1008](#)
 - middle of the list, [995–1002](#)
 - simple, [989–991](#)

- `LinkedList` class, [724–726](#), [841](#), [989–1002](#)
 - case study, [734–737](#)
- linked lists, [975–1027](#). See also [lists](#)
 - advantage of, [726](#)
 - as array list, [725](#)
 - benefits of, [730](#)
 - code details, [1017–1019](#)
 - construction of, [977–979](#)
 - defined, [725](#)
 - doubly, [1014](#)
 - iterators, [727–730](#), [1015–1017](#)
 - nodes, [725](#), [976–988](#)
 - variations, [1012–1014](#)
- list abstraction, [668](#)
- `listFiles` method, [791](#)
- `List` interface, [632](#), [731](#), [1141–1142](#)
- `ListIterator`, [730](#)
- lists, [723–737](#), [1141–1145](#)
 - capacity, [934](#)
 - circular, [1012–1013](#)
 - complex operations, [1002–1008](#)
 - defined, [723](#)
 - equivalents, [988](#)
 - linked. See [linked lists](#)
 - sets/maps vs., [753](#)
 - traversing, [985–988](#)
- literals, [68–69](#)
 - `boolean`, [69](#)

- characters, [69](#)
- defined, [65](#), [68](#)
- integers, [68](#)
- string, [14](#), [29](#)
- load factor, [1094](#)
- localizing variables, [103](#)
- local variables, [103](#), [150](#)
- logarithmic algorithm, [859](#)
- logical operators, [340–343](#), [1162](#)
 - AND (&&), [341](#)
 - defined, [340](#)
 - NOT (!), [341](#)
 - OR (| |), [341](#)
 - short-circuit evaluation, [343–348](#)
- logic errors, [24](#), [28](#)
- log-linear algorithm, [859](#)
- `long` type, [64](#)
- loops
 - `for`. See `for` [loop](#)
 - array, [467–468](#)
 - definite, [63](#), [321](#)
 - `do/while`, [331–333](#)
 - exiting, [1183–1184](#)
 - for-each, [467–468](#), [475](#), [684–686](#)
 - for graphics, [211–215](#)
 - indefinite, [63](#), [321](#)
 - infinite, [107](#), [324–325](#)
 - iterations, [95](#)

- `min/max`, [261–265](#)
- priming, [327–328](#)
- runtime(s), [853](#)
- sentinel, [337–339](#)
- traversing, [470](#), [474](#)
- `while`. See `while` loop
- loose coupling, [575](#)
- lottery case study, [743–745](#)

M

- machine language, **6**
- Macintosh computer, **538**
- magic numbers, **113**
- `main` method, **36**, **147**, **774**
 - as static method, **32**
 - syntax, **12**
- managing complexity, **103–115**
- Mandelbrot, Benoit, **797**
- `Map` interface, **632**
- `Map` objects, methods of, **1172**
- maps, **745–753**
 - case study, **750–753**
 - construction, **746–747**
 - defined, **723**, **746**
 - `keySet` method, **748–749**
 - key/value pair, **746–747**
 - methods of, **748**
 - operations, **746–747**
 - sets/lists vs., **753**
 - `TreeMap` vs. `HashMap`, **749–750**
 - `value` method, **748–749**
 - views, **748–749**
- `Math` class, **158–161**

- constants, [160](#)
- methods of, [1173](#)
- static methods in, [160–161](#)
- megabytes (MB), [5](#), [6](#)
- memory
 - defined, [3](#)
 - storage, units of, [6](#)
- merge sort
 - case study, [873–881](#)
 - defined, [873](#)
 - recursive, [876–879](#)
- merging arrays, [873–876](#)
- method call, [33](#)
- method headers, [12](#)
 - parameters in, [144](#)
 - stock fields and, [576–577](#)
- methods
 - `abs`, [161](#), [270](#), [1140–1141](#)
 - abstract, [634](#)
 - `add`, [670](#)
 - `addSorted`, [1002–1008](#)
 - `ArrayList`, [675](#)
 - `ArrayList` objects, [1168](#)
 - arrays and, [464–466](#)
 - `Arrays.asList`, [1141–1142](#)
 - `Arrays` class, [470](#)
 - `Arrays.stream`, [1140–1141](#)

- `binarySearch`, 842
- calling another method, 36–38
- `canRead`, 425
- `Character` class, 273, 1169
- `charAt`, 168, 285
- `checkIndex`, 953, 954
- `Collections` class, 732
- `Collections.sort`, 689
- `Color` object, 507
- `compare`, 848–849
- `compareTo`, 692, 848–849
- with conditional execution, 279–290
- `contains`, 675
- `copyOf`, 469
- `copyOfRange`, 469
- coupling, 303–304
- `deepToString`, 499
- defined, 11
- `doubleNumber`, 151
- `drawBox`, 33–36, 156–157
- `DrawingPanel` class, 202, 504, 1170
- `drawLine`, 203
- `drawOval`, 206, 208
- `drawRect`, 206, 208, 215
- `drawString`, 215, 220

- `drawText`, [220](#)
- `drawTriangle`, [774–776](#)
- `equals`, [256](#), [475–477](#), [610–613](#), [868](#)
- `equalsIgnoreCase`, [256](#)
- `Files.lines`, [1145](#)
- `fillOval`, [208](#)
- `fillPolygon`, [799](#)
- `fillRect`, [208](#), [220](#)
- `findNextToken`, [911](#)
- `get`, [673](#), [727](#), [747](#)
- `getBlue`, [507](#)
- `getGraphics`, [203](#), [629](#)
- `getGreen`, [507](#)
- `getInput`, [427](#)
- `getPixels`, [504–505](#)
- `getRed`, [507](#)
- `giveProblems`, [1124–1125](#)
- `Graphics2D`, [630](#)
- `graphics objects`, [1171](#)
- `hashCode`, [1095–1097](#)
- `hasNextInt`, [355](#), [356](#)
- `hasNextLine`, [414](#)
- `helper`, [794–797](#)
- `hypotenuse`, [164](#)
- `with indentation`, [793–794](#)

- `indexOf`, [284–285](#), [676–677](#), [841](#), [953](#)
- `instance`, [543–546](#)
- `IntStream.of`, [1132–1133](#)
- `IntStream.range`, [1130](#), [1131](#)
- `intValue`, [686](#)
- `isDirectory`, [790](#)
- `isEmpty`, [953](#), [1018](#)
- `Iterator` objects, [1172](#)
- `iterators`, [728](#)
- `keySet`, [748–749](#)
- `lastIndexOf`, [677](#)
- `length`, [543](#)
- `listFiles`, [791](#)
- `main`, [12](#), [32](#), [147](#), [774](#)
- `Map` objects, [1172](#)
- `maps`, [748](#)
- `Math` class, [160–161](#), [1173](#)
- `next`, [401](#)
- `nextDouble`, [175](#), [177](#), [179](#), [401](#), [405](#), [406](#)
- `nextInt`, [174](#), [179](#), [326](#), [401](#)
- `nextLine`, [174](#), [414](#)
- `Object` class, [609](#), [1174](#)
- `overloading`, [156–157](#)
- `overriding`, [600–602](#)
- `parallelSort`, [845](#)
- `Point` objects, [540](#)

- `pow`, 781, 783
- `print`, 790, 792–793
- `PriorityQueue<E>`, 1101
- `put`, 747
- `Queue` interface, 896
- `random`, 325
- `Random` objects, 326
- `range`, 855
- `rating`, 288
- `remove`, 727, 939
- `removeEvenLength`, 726
- `replaceAll`, 475
- return values, 157–164
- `reverse`, 772, 777–780
- `reverseOrder`, 851
- `setBackground`, 209–210
- `setColor`, 209–210, 213
- `setFont`, 216
- `setLocation`, 563
- `setPixels`, 504–505
- `shuffle`, 847
- signatures, 156–157
- `size`, 673
- `sort`, 845
- `sqrt`, 158

- `Stack` class, [894](#)
- `static`, [31–34](#), [153](#)
- `String` objects, [172](#)
- `substring`, [169–170](#), [344](#)
- `sum`, [162](#), [1130](#)
- `swap`, [479](#), [871](#)
- `takeDictation`, [597](#)
- `toLowerCase`, [171](#), [172](#)
- `toString`. See `toString` method
- `toUpperCase`, [171](#), [172](#)
- `translate`, [539](#), [544](#), [545–546](#)
- `value`, [748–749](#)
- `writeChars`, [153–154](#)
- `writeSpaces`, [143–146](#)
- `writeStars`, [767–769](#)
- mod operator (%), [70](#), [71](#)
- multidimensional arrays, [497–503](#). See also [arrays](#)
 - defined, [497](#)
 - jagged, [499–503](#)
 - rectangular two-dimensional, [497–499](#)
- multiple constructors, [559–560](#)
- multiple objects, [486–488](#)
- multiplication operator (*), [69–70](#)
- multiplicative operators, [73](#), [74](#), [88](#), [247](#)
- mutators, [548–549](#), [1122](#)

N

- namespaces, [1188](#)
- native compilers, [6](#)
- natural ordering, [691–694](#)
- nested `if/else` statements, [248–252](#)
 - final branch, [252](#)
 - flow of, [250](#), [251](#)
 - options, [253](#)
- nested loops
 - algorithms, [495–497](#)
 - code, [101](#)
 - defined, [100](#)
 - `for` loop, [100–102](#)
 - trace of, [108](#)
- *The New Hacker's Dictionary*, [38–39](#)
- `new` keyword, [174](#), [494](#)
- `nextDouble` method, [175](#), [177](#), [179](#), [401](#), [405–407](#)
- `nextInt` method, [174](#), [179](#), [326](#), [401](#)
- `nextLine` method, [174](#), [414](#)
- `next` method, [401](#)
- nodes, [976–988](#)
 - branch, [1029](#)
 - defined, [725](#), [976](#)
 - dummy, [1013](#)

- leaf, [1030–1031](#)
- manipulating, [982–985](#)
- and tree classes, [1032–1033](#)
- nonabstract classes, [651](#)
- nonprogramming hierarchies, [594–596](#)
- `NoSuchElementException`, [912](#), [960](#)
- not equal to (! =) operator, [246](#)
- NOT operator (!), [341](#)
- `NullPointerException`, [578](#), [615](#), [950](#), [1004](#), [1006](#)
- `NumberGuess` case study, [366–375](#)
- `number` parameter, [144](#), [148](#)
- numbers
 - binary, [4–5](#)
 - decimal, [4](#), [5](#)
 - magic, [113](#)
 - perfect, case study, [1146–1156](#)
 - pseudorandom, [325](#)
 - random, [325–329](#)
 - smallest divisor, finding, [322–323](#)

O

- `Object` class, **609–610**
 - methods of, **609, 1174**
- object-oriented design (OOD), **639, 640**
 - heuristics, **574–576**
- object-oriented programming (OOP), **485, 536–540, 594**
 - defined, **536**
- objects, **165–178**
 - array, **484, 493–494**
 - behavior, **536**
 - class, **166**
 - defined, **166, 536**
 - equality, **255–256**
 - exception, **282**
 - `File`, **396, 400**
 - `Graphics`, **203–208**
 - immutable, **171**
 - initialization, **552–560**
 - as instances, **166**
 - multiple, **486–488**
 - `Point`. See `Point` **objects**
 - `PrintStream`, **420–421**
 - `Random`, **326**
 - `Scanner`, **142, 173–178**

- searching, [868–869](#)
- state, [536](#), [541–543](#)
- `String`, [166–173](#)
- off-by-one bug, [462–463](#)
- offset, [222](#)
- OOD. See [object-oriented design \(OOD\)](#)
- OOP. See [object-oriented programming \(OOP\)](#)
- operands, [66](#), [819–820](#)
- operating system, [4](#)
 - history, and objects, [538](#)
- operators. See also specific operators
 - additive, [73](#), [74](#), [88](#), [247](#)
 - arithmetic, [69–72](#), [1161](#)
 - assignment, [83](#), [88](#), [247](#)
 - decrement (--), [87–90](#)
 - defined, [66](#)
 - increment (++), [87–90](#)
 - `instanceof`, [613](#)
 - logical. See [logical operators](#)
 - multiplicative, [73](#), [88](#), [247](#)
 - precedence, [72–74](#), [88](#), [247](#), [343](#), [1162](#)
 - relational, [246–248](#), [1162](#)
 - ternary, [1183](#)
 - unary, [74](#), [88](#), [247](#)
- OR operator (`|`), [258](#), [341–343](#)
 - precedence, [343](#)
 - short-circuited evaluation, [346](#)
 - truth table for, [341](#)

- output
 - cursor, [17](#), [18](#)
 - files, [420–424](#)
 - of program, [9](#)
- overloading methods, [156–157](#)
- overriding methods, [600–602](#)

P

- packages, **1188–1190**. See also specific packages
 - declaration syntax, **1189**
 - defined, **178**
- `parallelSort` method, **846**
- parameters, **143–157**
 - actual, **145, 149–150**
 - defined, **143**
 - formal, **145, 149–150**
 - implicit, **546–548**
 - initialization, **151**
 - limitations of, **150–152**
 - local manipulations of, **152**
 - mechanics of, **146–150**
 - in method headers, **144**
 - multiple, **153–155**
 - vs. constants, **156**
- parentheses (`()`), **93**
- Pascal's triangle, **500–501**
- paths
 - defined, **409**
 - files, **409–410**
 - reasoning about, **288–290**
 - relative, **410**
- PB (petabyte), **6**

- Peirce, Charles, [3](#)
- percent sign (%)
 - mod operator, [70](#), [71](#)
- percolating, [1103](#)
- perfect numbers case study, [1146–1156](#)
- petabyte (PB), [6](#)
- Pirsig, Robert, [350](#)
- pixels, [202](#)
 - arrays of, [504–509](#)
- `Point` class, [850](#)
 - encapsulation, [561–567](#), [612](#)
 - state and behavior, [540–546](#)
- `Point3D` class, [625–628](#)
- `Point` objects, [539–540](#), [545](#), [851](#)
 - constructing, [494](#)
 - defined, [539](#)
 - methods of, [540](#), [1176](#)
- `Polygon` class, [799](#)
- polymorphism
 - complex calls interpretation, [622–625](#)
 - inheritance and, [615–625](#)
 - inheritance code interpretation, [620–622](#)
 - mechanics, [618–620](#)
- pop operation, stack, [893](#)
- postconditions, [279](#)
- postdecrement (x--) operator, [88](#)
- postfix notation, [818–819](#)
- postincrement (x++) operator, [88](#)

- postorder traversals, [1034](#)
- `pow` method, [781](#), [783](#)
- precedence, [72–74](#), [88](#), [247](#), [343](#), [1162](#)
 - defined, [72](#)
 - logical operator, [342](#)
- preconditions, [279](#)
- predecrement (`--x`) operator, [88–89](#)
- prefix evaluator case study, [818–824](#)
- prefix notation, [818–819](#)
- preincrement (`++x`) operator, [88](#)
- preorder traversals, [1034](#)
- priming loops, [327](#)
- primitive types, [64–65](#), [616](#), [686](#), [1163](#), [1182](#)
- `printf` statement, [276](#), [692](#)
- printing
 - `ArrayIntList` class, [932–937](#)
 - `ArrayList`, [669](#)
 - arrays, [471–472](#)
- `println` statements, [16–18](#), [79](#), [471](#), [671](#)
 - in `toString` method, [551–552](#)
- `print` method, [790](#), [792–793](#)
- `print` statement, [17–18](#), [98](#)
- `PrintStream` class, [420–424](#)
- `PriorityQueue<E>` method, [1101](#)
- private access, [561–567](#), [604](#)
- `private` keyword, [561](#)
- probing, [1087](#), [1088](#)

- procedural decomposition
 - defined, [28](#)
 - example, [30](#)
 - flow of control, [34–36](#)
 - with graphics, [220–224](#)
 - static methods, [31–34](#)
- procedural design heuristics, [300–305](#)
- program errors, [24–28](#)
- programming
 - defensive, [283](#)
 - functional. See [functional programming](#)
 - Java environment, [8–10](#)
 - need for, [2–3](#)
 - object-oriented. See [object-oriented programming \(OOP\)](#)
 - process, [6–7](#)
- programs
 - array, [456–460](#)
 - `ArrayList`, [677–678](#)
 - chaining, [304–305](#)
 - client, [537](#)
 - cohesion, [302–303](#)
 - complexity. See [complexity](#)
 - defined, [3](#)
 - errors, [24–28](#)
 - execution, [6](#)
 - flow of control, [34–36](#)
 - Hello, world!, [10–14](#), [421–422](#)
 - interactive, [173–174](#)

- output of, **9**
- overview of, **10–23**
- performance, **852**
- readability, **21–23**
- projectile trajectory case study, **178–186**
- prompt, **177**
- protected access, **1190**
- provable assertion, **359–360**
- pruning, decision tree, **808**
- pseudocode, **108–113**, **357**, **367**, **795**, **804**
 - for backtracking solutions, **810–813**
 - defined, **108**
 - sort algorithm, **871–872**
- pseudorandom numbers, **325**
- `public` keyword, **33**
- push operation, stack, **893**
- `put` method, **747**
- pyramids case study, **224–230**
- Pythagorean theorem, **163**

Q

- quadratic algorithm, [859](#)
- quadratic probing, [1087](#)
- `Queue` interface, methods of, [896](#)
- queues
 - concepts, [896–897](#)
 - defined, [723](#), [896](#)
 - `Integer` wrapper class, [897–898](#)
 - operations, [897–898](#)
 - priority, [1099–1101](#)
 - removing values from, [904–906](#)
 - sum of, [900–901](#)
 - transfer values to stack, [899–900](#)
- `queueToStack` method, [902](#)

R

- RAM (random access memory), [3](#)
- random access, [461–462](#), [727](#)
- random access memory (RAM), [3](#)
- `random` method, [325](#)
- random numbers, [325–329](#)
- `Random` objects, [325–327](#)
 - methods of, [326](#), [1176](#)
 - misusing, [329](#)
- range algorithm, [855–858](#)
- `range` method, [855](#)
- `rating` method, [288](#)
- Raymond, Eric, [38](#)
- readability, program, [21–23](#)
- reading files, [393–400](#)
 - beyond end, [399–400](#)
 - with `Scanner` objects, [396–399](#)
 - structure of files, [403–407](#)
- reasoning, assertion, [360–362](#)
- records, [543](#), [604–605](#)
- `Rectangle` class, [635](#)
- rectangles, area and perimeter, [633](#)
- rectangular two-dimensional arrays, [497–499](#)
- recurrence relations, [867](#)

- recursion, [763–839](#)
 - backtracking, [801–818](#)
 - base case, [770–771](#)
 - binary search, [865–868](#)
 - bottom of, [766](#)
 - case study, [818–824](#)
 - data, [781–797](#)
 - defined, [764](#)
 - directories, [790–794](#)
 - example, [771–781](#)
 - file-reversing method, [776–781](#)
 - functions, [781–797](#)
 - graphics, [797–801](#)
 - helper methods, [794–797](#)
 - infinite, [770](#), [786](#), [787–789](#)
 - iterative solution converted to, [767–769](#)
 - mechanics of, [773–781](#)
 - merge sort, [876–879](#)
 - nonprogramming example, [764–767](#)
 - recursive case, [770–771](#)
 - solution structure, [769–771](#)
- recursive case, [770–771](#), [782](#), [785](#), [820](#)
- recursive merge sort, [876–879](#)
- `reduce`, stream terminator, [1134–1135](#)
- refactoring, [643–644](#)
- reference semantics, [484–488](#)
 - defined, [484–485](#)
 - multiple objects, [486–488](#)

- reasons for, [485–486](#)
- rehashing, [1092–1095](#)
- relational operators, [246–248](#), [1164](#)
 - limitation of, [248](#)
 - types, [247](#)
- relationships
 - has-a, [628](#)
 - is-a, [595](#), [598](#)
- relative file path, [409](#)
- remainder operator (%), [70–71](#)
- `removeEvenLength` method, [726](#)
- `remove` method, [727](#), [940](#)
- `replaceAll` method, [475](#)
- `return` statement, [161–162](#), [288](#)
- return type, [158](#)
- return values, [157–165](#)
 - ignoring, [163](#)
- `reverse` method, [772](#), [777–780](#)
- `reverseOrder` method, [851](#)
- RGB value, [504](#), [507](#)
- Ritchie, Dennis, [14](#)
- robust programs, [354](#)
- root, of binary tree, [1029](#), [1031–1032](#)
- roundoff errors, [267–270](#)
 - defined, [268](#)
 - outcomes, [268–269](#)
- `Runnable` interface, [632](#)

- running programs, **6**
- runtimes, **613**
 - of algorithm, **855–858**
 - binary search algorithm, **864–865**
 - errors, **24, 39–40**
 - of loop, **853**
 - statements, **853**

S

- `Scanner` class, **173–178, 910**
 - interactive program with, **176–178**
 - lookahead, **354–356**
- `Scanner` objects, **142**
 - in file processing, **407**
 - methods, **174, 1177**
 - parameters, **407–409**
 - reading files with, **396–399**
- scope, **103–107, 545**
 - defined, **103, 545**
 - implications, **103**
 - outer, **105**
- searching
 - `ArrayList`, **673–677**
 - arrays, **472–475**
 - binary, **842–845, 862–865, 867–868**
 - implementing algorithms, **861–872**
 - in Java's class libraries, **847**
 - libraries, **841–851**
 - objects, **868–869**
 - recursive binary, **865–867**
 - sequential search, **861–862**
- `SearchTree<E>`, **1067–1071**

- selection sort, **869–872**
- self-check problems
 - `ArrayLists`, **714–717**
 - arrays, **520–527**
 - classes, **583–585**
 - conditional execution, **306–314**
 - file processing, **435–444**
 - functional programming, **1157–1159**
 - graphics, **230–232**
 - Java Collections Framework, **754–758**
 - Java programming introduction, **47–54**
 - linked lists, **1020–1023**
 - parameters and objects, **187–195**
 - primitive data and definite loops, **124–132**
 - program logic and indefinite loops, **376–386**
 - recursion, **825–831**
 - searching and sorting, **883–887**
 - stacks and queues, **920–924**
- semicolon (;), **12, 15, 26–27**
- sentinel loops, **337–339**
- separate chaining, **1088–1092**
- sequences
 - escape, **15–16**
 - hailstone, **263**
- sequential access, **461**
- sequential algorithms, **447**
- sequential search, **841**
 - of array, **861–862**

- `Serializable` interface, [632](#)
- `setBackground` method, [209–210](#)
- `Set` collection, [737](#)
- `setColor` method, [209–210](#), [213](#)
- `setFont` method, [216](#)
- `Set` interface, [632](#)
- `setLocation` method, [563](#)
- `setPixels` method, [504–505](#)
- sets, [737–745](#)
 - case study, [743–745](#)
 - concepts, [738–740](#)
 - defined, [723](#), [737](#)
 - maps/lists vs., [753](#)
 - operations, [741–743](#)
 - `TreeSet` vs. `HashSet`, [738–741](#)
- shapes
 - area and perimeter formulas, [633](#)
 - drawing, [203–208](#)
 - interfaces for, [632–634](#)
- `ShareAsset` class, [645–646](#)
- short-circuit evaluation, [343–348](#)
- `short` type, [64](#)
- `shuffle` method, [846](#)
- shuffling, [846–847](#)
- side effect, [1121](#)
- Sierpinski triangle, [797–798](#)
- sieve algorithm case study, [734–737](#)

- signatures, method, **156–157**
- simulations, **329–331**
- single inheritance, **598**
- `size` method, **673**
- smallest divisor, **322–323**
- software, **4**
 - crisis, **593**
- sorted array, **1084**
- sorting, **845–846, 852**
 - `Arrays.parallelSort` method, **845**
 - `Arrays.sort` method, **845**
 - `Collections.sort` method, **846**
 - implementing algorithms, **861–872**
 - in Java's class libraries, **847**
 - selection sort, **869–872**
- `sort` method, **845**
- spaces
 - `String` objects, **168**
 - writing, **143–146**
- space-writing task, **143–144**
- specifiers, format, **276–277**
- splitting arrays, **873–876**
- `sqrt` method, **158–159**
- stack, **893–895**
 - comparison for similarity, **906–908**
 - defined, **723, 894**
 - `Integer` wrapper class, **897–898**

- operations, [897–904](#)
- sum of, [901–904](#)
- transfer values to queue, [899–900](#)
- `Stack` class, methods of, [894](#)
- stack trace, [170](#)
- state, objects, [541–543](#)
 - defined, [536](#)
- statements
 - `assert`, [1187](#)
 - assignment, [79–83](#)
 - `break`, [1185–1186](#)
 - `continue`, [1183–1184](#)
 - defined, [12](#)
 - `if/else`, [772–773](#). See `if/else` [statements](#)
 - `print`, [17–18](#), [98](#)
 - `printf`, [707](#)
 - `println`, [16–18](#), [79](#), [671](#)
 - prompt, [177](#)
 - `return`, [161–162](#), [288](#)
 - runtimes, [853](#)
 - `switch`, [1184–1186](#)
 - `throw`, [281](#)
 - `try/catch`, [1186–1187](#)
- `static` keyword, [33](#), [544](#)
- static methods, [31–34](#)
 - defined, [32](#)

- in `Math` class, **160–161**
- structure, **32**
- syntax, **153**
- Steele, Guy, **1122**
- stepwise refinement, **31**
- `Stock` class case study, **573–582**
- streams, **1129–1136**
 - defined, **1129**
 - using filter, **1132–1134**
 - using map, **1131–1132**
 - using `reduce`, **1134–1135**
- `String` class, **693–694**
- string concatenation, **84–87**
- `StringIndexOutOfBoundsException` Exception, **347**
- string literals, **14, 29**
- `String` objects, **166–173**
 - index, **168**
 - length, **167–168**
 - methods of, **172, 1177**
 - spaces, **168**
- strings
 - defined, **14**
 - format, **276**
 - line breaks inside, **15–16**
 - programming error, **29**
 - traversal algorithm, **481–482**
- struct, **543**
- subclasses, **598**. See also **inheritance**

- subset, [742](#)
- substitutability, [627](#)
- `substring` method, [169–170](#), [344](#), [1130](#)
- `sum` method, [162](#), [1130](#)
- superclasses. See also [inheritance](#)
 - accessing inherited fields, [603–605](#)
 - constructors, calling, [605–607](#)
 - declaration syntax, [598](#)
 - defined, [598](#)
 - interacting with, [602–615](#)
 - overridden methods, [602–603](#)
- superset, [742](#)
- Sussman, Gerry, [1122](#)
- `swap` method, [479](#), [871](#)
- `switch` statement, [1172–1174](#)
- syntax
 - abstract classes, [648](#)
 - abstract method declaration, [649](#)
 - array declaration and construction, [449](#)
 - array initialization, [455–456](#)
 - constant definitions, [114](#)
 - constructors, [555](#)
 - `do/while` loops, [331](#)
 - encapsulated field declaration, [562](#)
 - errors, [24–28](#)
 - for-each loop, [467–468](#)
 - instance methods, [546](#)
 - interface declaration, [634](#)

- `for` loop, **93**
- packages declaration, **1189**
- `return` statement, **162**
- static method declaration, **153**
- superclass declaration, **598**
- template, **11**
- templates, **1163–1167**
- `this` keyword, **557–559**
- `throw` statement, **281**
- `while` loop, **321**
- `System.currentTimeMillis` method, **1150–1151**
- `System.out.print`, **173**
- `System.out.printf`, **274–278**
- `System.out.println`, **172**

T

- tags, Javadoc comments, **1181**
- `takeDictation` method, **597**
- tallying values, **510–514**
- terabyte (TB), **6**
- terminal interaction, **9**
- ternary operator, **1183**
- text
 - cumulative algorithms, **272–274**
 - files, **395**
 - processing, **270–278**
- `this` keyword, **557–559**
- throwing exceptions, **170**, **279–283**, **949–952**
- `throws` clause, **398**, **425**
- `throw` statement, **281**
- time complexity, **852**
- `TimeSpan` class, **571**
- token-based processing, **401–413**. See also **files**
 - defined, **401**
 - input file, **412–413**
 - line-based combination, **415–419**
 - paths and directories, **409–411**
 - `Scanner` parameters, **407–409**
 - structure of files, **403–407**

- tokens
 - defined, [174](#)
 - reading wrong kind of, [404](#)
 - splitting, [909–914](#)
 - whitespace-separated, [405](#)
- `toLowerCase` method, [171–172](#), [1143](#)
- `toString` method, [469](#), [499](#), [671](#), [738](#), [747](#), [895](#), [936](#), [994](#)
 - implementing, [551](#)
 - `println` statement in, [551–552](#)
- `toUpperCase` method, [171–172](#)
- `translate` method, [539](#), [544](#), [545–546](#)
- traversing loops, [470](#), [474](#)
- `TreeMap` class, [746](#), [850](#), [1053](#)
 - vs. `HashMap` class, [749–750](#)
- `TreeSet` class, [723](#), [738–741](#), [850](#), [1053](#)
- tree traversals, [1033–1048](#). See also [binary tree](#)
 - constructing and viewing, [1039–1048](#)
 - preorder/postorder/inorder, [1034](#)
- triangle, area and perimeter, [633](#)
- `Triangle` class, [637](#)
- `try/catch` statement, [1186–1187](#)
- two-dimensional arrays, [497–499](#)
- types
 - `boolean`, [64](#), [69](#), [320](#), [339–354](#)
 - `byte`, [64](#)
 - `char`, [64](#)

- `double`, [64–65](#)
- `float`, [64–65](#)
- `int`, [64](#), [271–272](#)
- `long`, [64](#)
- mixing, [74–76](#)
- primitive, [64–65](#)
- `short`, [64](#)
- type-safe language, [64](#)

U

- unary operators, [74](#), [88](#), [247](#)
- unboxing, [688](#), [690–691](#)
- underscore (`_`) character, [19](#)
- Unicode, [275](#)
- uninitialized variables, [78](#)
- union operation, [742](#)
- unsorted array, [1084](#)
- user errors, [354–358](#)

V

- `value` method, [748–749](#)
- values
 - adding to `ArrayList`, [671–672](#)
 - comparison function, [691](#)
 - natural ordering, [691–694](#)
 - removing from `ArrayList`, [672](#)
 - semantics, [484–485](#)
 - shifting in arrays, [488–493](#)
 - tallying, [510–514](#)
 - types, [484](#)
- value semantics, [485](#)
- variables, [76–92](#)
 - assigning, [78–80](#)
 - assignment/declaration variations, [81–84](#)
 - `boolean` type, [339](#), [348–350](#)
 - bound, [1137–1138](#)
 - declarations, [77–78](#)
 - decrementing, [81](#), [87](#)
 - defined, [76](#)
 - free, [1137–1138](#)
 - global, [1121](#)
 - incrementing, [81](#), [87](#)
 - initial value, [81](#)

- local, [103](#), [150](#)
- localizing, [103](#)
- loop, [95](#)
- mixing types and, [90–91](#)
- in `println` statement, [79](#)
- uninitialized, [78](#)
- Venn diagrams, [742](#)
- verification, program, [360](#)
- `void` keyword, [33](#), [555–556](#)

W

- `while` loop, [321–333](#). See also [loops](#)
 - defined, [320](#)
 - `do/while` loops, [331–333](#)
 - flow of, [321](#)
 - priming, [327](#)
 - random numbers, [325–328](#)
 - simulations, [329–331](#)
 - smallest divisor, [322–323](#)
 - syntax of, [321](#)
- whitespace, [174](#), [405](#)
- `WordCount` case study, [750–753](#)
- working directory, [409](#)
- worst case analysis, [862](#)
- wrapper classes, [686–688](#), [1186](#)
 - boxing, [688](#)
 - common, [688](#)
 - defined, [686](#)
 - unboxing, [688](#)
- `writeChars` methods, [153–154](#)
- `writeSpaces` method, [143–145](#)
- `writeStars` method, [767–769](#)

Y

- Y2K (millennium) bug, [564](#)

Z

- Zen, Boolean, [350–352](#)
- zero-based indexing, [448](#), [671](#)
- zip code lookup case study, [427–434](#)

Contents

1. **Building Java Programs A Back to Basics Approach**
2. **Preface**
 - A. **New to This Edition**
 - B. **Features from Prior Editions**
 - C. **Layers and Dependencies**
 - D. **Supplements**
 - E. **MyLab Programming**
 - F. **VideoNotes**

Location of Video Notes in the Text

3. <http://www.pearson.com/cs-resources>
4. **Brief Contents**
5. **Contents**
6. **Chapter 1 Introduction to Java Programming**
 - A. **Introduction**
 - B. **1.1 Basic Computing Concepts**
 1. **Why Programming?**
 2. **Hardware and Software**
 3. **The Digital Realm**
 4. **The Process of Programming**
 5. **Why Java?**
 6. **The Java Programming Environment**
 - C. **1.2 And Now—Java**

1. **String Literals (Strings)**
2. `System.out.println`
3. **Escape Sequences**
4. `print` versus `println`
5. **Identifiers and Keywords**
6. **A Complex Example:** `DrawFigures1`
7. **Comments and Readability**

D. **1.3 Program Errors**

1. **Syntax Errors**
2. **Logic Errors (Bugs)**

E. **1.4 Procedural Decomposition**

1. **Static Methods**
2. **Flow of Control**
3. **Methods That Call Other Methods**
4. **An Example Runtime Error**

F. **1.5 Case Study:** `DrawFigures`

1. **Structured Version**
2. **Final Version without Redundancy**
3. **Analysis of Flow of Execution**

G. **Chapter Summary**

H. **Self-Check Problems**

1. **Section 1.1: Basic Computing Concepts**
2. **Section 1.2: And Now—Java**
3. **Section 1.3: Program Errors**

4. Section 1.4: Procedural Decomposition

I. Exercises

J. Programming Projects

7. Chapter 2 Primitive Data and Definite Loops

A. Introduction

B. 2.1 Basic Data Concepts

1. Primitive Types
2. Expressions
3. JShell
4. Literals
5. Arithmetic Operators
6. Precedence
7. Mixing Types and Casting

C. 2.2 Variables

1. Assignment/Declaration Variations
2. String Concatenation
3. Increment/Decrement Operators
4. Variables and Mixing Types

D. 2.3 The `for` Loop

1. Tracing `for` Loops
2. `for` Loop Patterns
3. Nested `for` Loops

E. 2.4 Managing Complexity

1. **Scope**
2. **Pseudocode**
3. **The Table Technique**
4. **Class Constants**

F. **2.5 Case Study: Hourglass Figure**

1. **Problem Decomposition and Pseudocode**
2. **Initial Structured Version**
3. **Adding a Class Constant**
4. **Further Variations**

G. **Chapter Summary**

H. **Self-Check Problems**

1. **Section 2.1: Basic Data Concepts**
2. **Section 2.2: Variables**
3. **Section 2.3: The `for` Loop**
4. **Section 2.4: Managing Complexity**

I. **Exercises**

J. **Programming Projects**

8. **Chapter 3 Introduction to Parameters and Objects**

A. **Introduction**

B. **3.1 Parameters**

1. **The Mechanics of Parameters**
2. **Limitations of Parameters**
3. **Multiple Parameters**
4. **Parameters versus Constants**

5. Overloading of Methods

C. 3.2 Methods That Return Values

1. The `Math` Class
2. Defining Methods That Return Values

D. 3.3 Using Objects

1. `String` Objects
2. Interactive Programs and Scanner Objects
3. Sample Interactive Program

E. 3.4 Case Study: Projectile Trajectory

1. Unstructured Solution
2. Structured Solution

F. Chapter Summary

G. Self-Check Problems

1. Section 3.1: Parameters
2. Section 3.2: Methods That Return Values
3. Section 3.3: Using Objects

H. Exercises

I. Programming Projects

9. Supplement 3G Graphics (Optional)

A. Introduction

B. 3G.1 Introduction to Graphics

1. `DrawingPanel`

2. **Drawing Lines and Shapes**
3. **Colors**
4. **Drawing with Loops**
5. **Text and Fonts**
6. **Images**

C. **3G.2 Procedural Decomposition with Graphics**

1. **A Larger Example:** `DrawDiamonds`

D. **3G.3 Case Study: Pyramids**

1. **Unstructured Partial Solution**
2. **Generalizing the Drawing of Pyramids**
3. **Complete Structured Solution**

E. **Chapter Summary**

F. **Self-Check Problems**

1. **Section 3G.1: Introduction to Graphics**

G. **Exercises**

H. **Programming Projects**

10. **Chapter 4 Conditional Execution**

A. **Introduction**

B. **4.1 `if/else` Statements**

1. **Relational Operators**
2. **Nested `if/else` Statements**
3. **Object Equality**
4. **Factoring `if/else` Statements**

5. Testing Multiple Conditions

C. 4.2 Cumulative Algorithms

1. Cumulative Sum
2. Min/Max Loops
3. Cumulative Sum with `if`
4. Roundoff Errors

D. 4.3 Text Processing

1. The `char` Type
2. `char` versus `int`
3. Cumulative Text Algorithms
4. `System.out.printf`

E. 4.4 Methods with Conditional Execution

1. Preconditions and Postconditions
2. Throwing Exceptions
3. Revisiting Return Values
4. Reasoning about Paths

F. 4.5 Case Study: Body Mass Index

1. One-Person Unstructured Solution
2. Two-Person Unstructured Solution
3. Two-Person Structured Solution
4. Procedural Design Heuristics

G. Chapter Summary

H. Self-Check Problems

1. **Section 4.1: `if/else` Statements**
2. **Section 4.2: Cumulative Algorithms**
3. **Section 4.3: Text Processing**
4. **Section 4.4: Methods with Conditional Execution**

I. **Exercises**

J. **Programming Projects**

11. **Chapter 5 Program Logic and Indefinite Loops**

A. **Introduction**

B. **5.1 The `while` Loop**

1. **A Loop to Find the Smallest Divisor**
2. **Random Numbers**
3. **Simulations**
4. **`do/while` Loop**

C. **5.2 Fencepost Algorithms**

1. **Fencepost with `if`**
2. **Sentinel Loops**

D. **5.3 The `boolean` Type**

1. **Logical Operators**
2. **Short-Circuited Evaluation**
3. **`boolean` Variables and Flags**
4. **Boolean Zen**
5. **Negating Boolean Expressions**

E. **5.4 User Errors**

1. `Scanner` Lookahead
2. Handling User Errors

F. **5.5 Assertions and Program Logic**

1. Reasoning about Assertions
2. A Detailed Assertions Example

G. **5.6 Case Study: `NumberGuess`**

1. Initial Version without Hinting
2. Randomized Version with Hinting
3. Final Robust Version

H. **Chapter Summary**

I. **Self-Check Problems**

1. Section 5.1: The `while` Loop
2. Section 5.2: Fencepost Algorithms
3. Section 5.3: The boolean Type
4. Section 5.4: User Errors
5. Section 5.5: Assertions and Program Logic

J. **Exercises**

K. **Programming Projects**

12. **Chapter 6 File Processing**

A. **Introduction**

B. **6.1 File-Reading Basics**

1. Data, Data Everywhere
2. Files and File Objects

3. **Reading a File with a `Scanner`**

C. **6.2 Details of Token-Based Processing**

1. **Structure of Files and Consuming Input**
2. **Scanner Parameters**
3. **Paths and Directories**
4. **A More Complex Input File**

D. **6.3 Line-Based Processing**

1. **`String` `Scanners` and Line/Token Combinations**

E. **6.4 Advanced File Processing**

1. **Output Files with `PrintStream`**
2. **Guaranteeing That Files Can Be Read**

F. **6.5 Case Study: Zip Code Lookup**

G. **Chapter Summary**

H. **Self-Check Problems**

1. **Section 6.1: File-Reading Basics**
2. **Section 6.2: Details of Token-Based Processing**
3. **Section 6.3: Line-Based Processing**
4. **Section 6.4: Advanced File Processing**

I. **Exercises**

J. **Programming Projects**

13. **Chapter 7 Arrays**

A. **Introduction**

B. 7.1 Array Basics

1. **Constructing and Traversing an Array**
2. **Accessing an Array**
3. **Initializing Arrays**
4. **A Complete Array Program**
5. **Random Access**
6. **Arrays and Methods**
7. **The For-Each Loop**
8. **The `Arrays` Class**

C. 7.2 Array-Traversal Algorithms

1. **Printing an Array**
2. **Searching and Replacing**
3. **Testing for Equality**
4. **Reversing an Array**
5. **String Traversal Algorithms**
6. **Functional Approach**

D. 7.3 Reference Semantics

1. **Multiple Objects**

E. 7.4 Advanced Array Techniques

1. **Shifting Values in an Array**
2. **Arrays of Objects**
3. **Command-Line Arguments**
4. **Nested Loop Algorithms**

F. 7.5 Multidimensional Arrays

1. **Rectangular Two-Dimensional Arrays**
2. **Jagged Arrays**

G. **7.6 Arrays of Pixels**

H. **7.7 Case Study: Benford's Law**

1. **Tallying Values**
2. **Completing the Program**

I. **Chapter Summary**

J. **Self-Check Problems**

1. **Section 7.1: Array Basics**
2. **Section 7.2: Array-Traversal Algorithms**
3. **Section 7.3: Reference Semantics**
4. **Section 7.4: Advanced Array Techniques**
5. **Section 7.5: Multidimensional Arrays**

K. **Exercises**

L. **Programming Projects**

14. **Chapter 8 Classes**

A. **Introduction**

B. **8.1 Object-Oriented Programming**

1. **Classes and Objects**
2. `Point` **Objects**

C. **8.2 Object State and Behavior**

1. **Object State: Fields**
2. **Object Behavior: Methods**

3. **The Implicit Parameter**
4. **Mutators and Accessors**
5. **The `toString` Method**

D. **8.3 Object Initialization: Constructors**

1. **The Keyword `this`**
2. **Multiple Constructors**

E. **8.4 Encapsulation**

1. **Private Fields**
2. **Class Invariants**
3. **Changing Internal Implementations**

F. **8.5 Case Study: Designing a `Stock` Class**

1. **Object-Oriented Design Heuristics**
2. **`Stock` Fields and Method Headers**
3. **`Stock` Method and Constructor Implementation**

G. **Chapter Summary**

H. **Self-Check Problems**

1. **Section 8.1: Object-Oriented Programming**
2. **Section 8.2: Object State and Behavior**
3. **Section 8.3: Object Initialization: Constructors**
4. **Section 8.4: Encapsulation**
5. **Section 8.5: Case Study: Designing a Stock Class**

I. **Exercises**

J. **Programming Projects**

15. Chapter 9 Inheritance and Interfaces

A. Introduction

B. 9.1 Inheritance Basics

1. Nonprogramming Hierarchies
2. Extending a Class
3. Overriding Methods

C. 9.2 Interacting with the Superclass

1. Calling Overridden Methods
2. Accessing Inherited Fields
3. Calling a Superclass's Constructor
4. `DividendStock` Behavior
5. The `Object` Class
6. The `equals` Method
7. The `instanceof` Keyword

D. 9.3 Polymorphism

1. Polymorphism Mechanics
2. Interpreting Inheritance Code
3. Interpreting Complex Calls

E. 9.4 Inheritance and Design

1. A Misuse of Inheritance
2. Is-a Versus Has-a Relationships
3. `Graphics2D`

F. 9.5 Interfaces

1. An Interface for Shapes

2. **Implementing an Interface**
3. **Benefits of Interfaces**

G. **9.6 Case Study: Financial Class Hierarchy**

1. **Designing the Classes**
2. **Redundant Implementation**
3. **Abstract Classes**

H. **Chapter Summary**

I. **Self-Check Problems**

1. **Section 9.1: Inheritance Basics**
2. **Section 9.2: Interacting with the Superclass**
3. **Section 9.3: Polymorphism**
4. **Section 9.4: Inheritance and Design**
5. **Section 9.5: Interfaces**
6. **Section 9.6: Case Study: Financial Class Hierarchy**

J. **Exercises**

K. **Programming Projects**

16. **Chapter 10 ArrayLists**

A. **Introduction**

B. **10.1** `ArrayLists`

1. **Basic** `ArrayList` **Operations**
2. `ArrayList` **Searching Methods**
3. **A Complete** `ArrayList` **Program**
4. **Adding to and Removing from an** `ArrayList`
5. **Initializing an** `ArrayList`

6. Using the For-Each Loop with `ArrayList`s
7. Wrapper Classes

C. **10.2 The `Comparable` Interface**

1. Natural Ordering and `compareTo`
2. Implementing the `Comparable` Interface

D. **10.3 Case Study: Ranked Choice Voting**

1. Ballot Class
2. Counting Votes
3. Multiple Rounds

E. **Chapter Summary**

F. **Self-Check Problems**

1. **Section 10.1:** `ArrayLists`
2. **Section 10.2:** The `Comparable` Interface

G. **Exercises**

H. **Programming Projects**

17. **Chapter 11 Java Collections Framework**

A. **Introduction**

B. **11.1 Lists**

1. Collections
2. `LinkedList` versus `ArrayList`
3. Iterators
4. Abstract Data Types (ADTs)

5. `LinkedList` **Case Study:** `Sieve`

C. **11.2 Sets**

1. **Set Concepts**
2. `TreeSet` **versus** `HashSet`
3. **Set Operations**
4. **Set Case Study:** `Lottery`

D. **11.3 Maps**

1. **Basic Map Operations**
2. **Map Views** (`keySet` and `values`)
3. `TreeMap` **versus** `HashMap`
4. **Map Case Study:** `WordCount`
5. **Collection Overview**

E. **Chapter Summary**

F. **Self-Check Problems**

1. **Section 11.1: Lists**
2. **Section 11.2: Sets**
3. **Section 11.3: Maps**

G. **Exercises**

H. **Programming Projects**

18. **Chapter 12 Recursion**

A. **Introduction**

B. **12.1 Thinking Recursively**

1. **A Nonprogramming Example**

2. **An Iterative Solution Converted to Recursion**
 3. **Structure of Recursive Solutions**
- C. **12.2 A Better Example of Recursion**
1. **Mechanics of Recursion**
- D. **12.3 Recursive Functions and Data**
1. **Integer Exponentiation**
 2. **Greatest Common Divisor**
 3. **Directory Crawler**
 4. **Helper Methods**
- E. **12.4 Recursive Graphics**
- F. **12.5 Recursive Backtracking**
1. **A Simple Example: Traveling North/East**
 2. **8 Queens Puzzle**
 3. **Solving Sudoku Puzzles**
- G. **12.6 Case Study: Prefix Evaluator**
1. **Infix, Prefix, and Postfix Notation**
 2. **Evaluating Prefix Expressions**
 3. **Complete Program**
- H. **Chapter Summary**
- I. **Self-Check Problems**
1. **Section 12.1: Thinking Recursively**
 2. **Section 12.2: A Better Example of Recursion**
 3. **Section 12.3: Recursive Functions and Data**

4. **Section 12.4: Recursive Graphics**
5. **Section 12.5: Recursive Backtracking**

J. **Exercises**

K. **Programming Projects**

19. **Chapter 13 Searching and Sorting**

A. **Introduction**

B. **13.1 Searching and Sorting in the Java Class Libraries**

1. **Binary Search**
2. **Sorting**
3. **Shuffling**
4. **Custom Ordering with Comparators**

C. **13.2 Program Complexity**

1. **Empirical Analysis**
2. **Complexity Classes**

D. **13.3 Implementing Searching and Sorting Algorithms**

1. **Sequential Search**
2. **Binary Search**
3. **Recursive Binary Search**
4. **Searching Objects**
5. **Selection Sort**

E. **13.4 Case Study: Implementing Merge Sort**

1. **Splitting and Merging Arrays**
2. **Recursive Merge Sort**

3. Complete Program

F. Chapter Summary

G. Self-Check Problems

Section 13.1: Searching and Sorting in the Java Class

1. Libraries

2. Section 13.2: Program Complexity

Section 13.3: Implementing Searching and Sorting

3. Algorithms

4. Section 13.4: Case Study: Implementing Merge Sort

H. Exercises

I. Programming Projects

20. Chapter 14 Stacks and Queues

A. Introduction

B. 14.1 Stack/Queue Basics

1. Stack Concepts

2. Queue Concepts

C. 14.2 Common Stack/Queue Operations

1. Transferring between Stacks and Queues

2. Sum of a Queue

3. Sum of a Stack

D. 14.3 Complex Stack/Queue Operations

1. Removing Values from a Queue

2. Comparing Two Stacks for Similarity

E. **14.4 Case Study: Expression Evaluator**

1. **Splitting into Tokens**
2. **The Evaluator**

F. **Chapter Summary**

G. **Self-Check Problems**

1. **Section 14.1: Stack/Queue Basics**
2. **Section 14.2: Common Stack/Queue Operations**
3. **Section 14.3: Complex Stack/Queue Operations**

H. **Exercises**

I. **Programming Projects**

21. **Chapter 15 Implementing a Collection Class**

A. **Introduction**

B. **15.1 Simple `ArrayIntList`**

1. **Adding and Printing**
2. **Thinking about Encapsulation**
3. **Dealing with the Middle of the List**
4. **Another Constructor and a Constant**
5. **Preconditions and Postconditions**

C. **15.2 A More Complete `ArrayIntList`**

1. **Throwing Exceptions**
2. **Convenience Methods**

D. **15.3 Advanced Features**

1. **Resizing When Necessary**

2. Adding an Iterator

E. 15.4 `ArrayList<E>`

F. Chapter Summary

G. Self-Check Problems

1. Section 15.1: Simple `ArrayIntList`
2. Section 15.2: A More Complete `ArrayIntList`
3. Section 15.3: Advanced Features
4. Section 15.4: `ArrayList<E>`

H. Exercises

I. Programming Projects

22. Chapter 16 Linked Lists

A. Introduction

B. 16.1 Working with Nodes

1. Constructing a List
2. List Basics
3. Manipulating Nodes
4. Traversing a List

C. 16.2 A Linked List Class

1. Simple `LinkedList`
2. Appending `add`
3. The Middle of the List

D. 16.3 A Complex List Operation

1. Inchworm Approach

E. **16.4 An `IntList` Interface**

F. **16.5 `LinkedList<E>`**

1. **Linked List Variations**
2. **Linked List Iterators**
3. **Other Code Details**

G. **Chapter Summary**

H. **Self-Check Problems**

1. **Section 16.1: Working with Nodes**
2. **Section 16.2: A Linked List Class**
3. **Section 16.3: A Complex List Operation**
4. **Section 16.4: An `IntList` Interface**
5. **Section 16.5: `LinkedList<E>`**

I. **Exercises**

J. **Programming Projects**

23. **Chapter 17 Binary Trees**

A. **Introduction**

B. **17.1 Binary Tree Basics**

1. **Node and Tree Classes**

C. **17.2 Tree Traversals**

1. **Constructing and Viewing a Tree**

D. **17.3 Common Tree Operations**

1. **Sum of a Tree**
2. **Counting Levels**

3. Counting Leaves

E. 17.4 Binary Search Trees

1. The Binary Search Tree Property
2. Building a Binary Search Tree
3. The Pattern `x = change(x)`
4. Searching the Tree
5. Binary Search Tree Complexity

F. 17.5 `SearchTree<E>`

G. Chapter Summary

H. Self-Check Problems

1. Section 17.1: Binary Tree Basics
2. Section 17.2: Tree Traversals
3. Section 17.3: Common Tree Operations
4. Section 17.4: Binary Search Trees
5. Section 17.5: `SearchTree<E>`

I. Exercises

J. Programming Projects

24. Chapter 18 Advanced Data Structures

A. Introduction

B. 18.1 Hashing

1. Array Set Implementations
2. Hash Functions and Hash Tables
3. Collisions
4. Rehashing

5. **Hashing Non-Integer Data**
6. **Hash Map Implementation**

C. **18.2 Priority Queues and Heaps**

1. **Priority Queues**
2. **Introduction to Heaps**
3. **Removing from a Heap**
4. **Adding to a Heap**
5. **Array Heap Implementation**
6. **Heap Sort**

D. **Chapter Summary**

E. **Self-Check Problems**

1. **Section 18.1: Hashing**
2. **Section 18.2: Priority Queues and Heaps**

F. **Exercises**

G. **Programming Projects**

25. **Chapter 19 Functional Programming with Java 8**

A. **Introduction**

B. **19.1 Effect-Free Programming**

C. **19.2 First-Class Functions**

1. **Lambda Expressions**

D. **19.3 Streams**

1. **Basic Idea**
2. **Using Map**

3. **Using Filter**
4. **Using Reduce**
5. **Optional Results**

E. **19.4 Function Closures**

F. **19.5 Higher-Order Operations on Collections**

1. **Working with Arrays**
2. **Working with Lists**
3. **Working with Files**

G. **19.6 Case Study: Perfect Numbers**

1. **Computing Sums**
2. **Incorporating Square Root**
3. **Just Five and Leveraging Concurrency**

H. **Chapter Summary**

I. **Self-Check Problems**

1. **19.1 Effect-Free Programming**
2. **19.2 First-Class Functions**
3. **19.3 Streams**
4. **19.4 Function Closures**
5. **19.5 Higher-Order Operations on Collections**

J. **Exercises**

K. **Programming Projects**

26. **Appendix A Java Summary**

A. **Java Keywords**

- B. **Primitive Types**
- C. **Arithmetic Operators**
- D. **Relational Operators**
- E. **Logical Operators**
- F. **Operator Precedence**
- G. **Wrapper Classes**
- H. **Syntax Templates**
 - I. **Useful Methods of `ArrayList` Objects**
 - J. **Useful Methods of the `Character` Class**
 - K. **Useful Methods of the `Collection` Interface**
 - L. **Useful Methods of the `Collections` Class**
 - M. **Useful Methods of `DrawingPanel` Objects**
 - N. **Useful Methods of `File` Objects**
 - O. **Useful Methods of `Graphics` Objects**
 - P. **Useful Methods of `Iterator` Objects**
 - Q. **Useful Methods of `Map` Objects**
 - R. **Constants and Useful Methods of the `Math` Class**
 - S. **Useful Methods of the `Object` Class**
 - T. **Useful Methods of `Point` Objects**
 - U. **Useful Methods of `Random` Objects**
 - V. **Useful Methods of `Scanner` Objects**
 - W. **Useful Methods of `String` Objects**

Appendix B The Java API Specification and Javadoc

27. Comments

- A. **The Java API Specification**

1. **Parameters**
2. **Returns**
3. **Throws**
4. **Deprecated**

B. **Writing Javadoc Comments**

28. **Appendix C Additional Java Syntax**

- A. **Primitive Types:** `byte`, `short`, `long`, `float`
- B. **Ternary Operator ? :**
- C. **Exiting a Loop:** `break` and `continue`
- D. **The `switch` Statement**
- E. **The `try/ catch` Statement**
- F. **The `assert` Statement**
- G. **Enumerations:** `enum`
- H. **Packages**
 - I. **Protected and Default Access**

29. **Index**

- A. **A**
- B. **B**
- C. **C**
- D. **D**
- E. **E**
- F. **F**
- G. **G**
- H. **H**
- I. **I**

J. **J**
K. **K**
L. **L**
M. **M**
N. **N**
O. **O**
P. **P**
Q. **Q**
R. **R**
S. **S**
T. **T**
U. **U**
V. **V**
W. **W**
X. **Y**
Y. **Z**

List of Illustrations

Landmarks

1. **Brief Contents**
2. **Frontmatter**
3. **Start of Content**
4. **backmatter**

5. List of Illustrations

1. **i**
2. **ii**
3. **iii**
4. **iv**
5. **v**
6. **vi**
7. **vii**
8. **viii**
9. **ix**
10. **x**
11. **xi**
12. **xii**
13. **xiii**
14. **xiv**
15. **xv**
16. **xvi**
17. **xvii**
18. **xviii**
19. **xix**
20. **xx**
21. **xxi**
22. **xxii**
23. **xxiii**
24. **xxiv**
25. **xxv**
26. **xxvi**

- 27. **xxvii**
- 28. **xxviii**
- 29. **1**
- 30. **2**
- 31. **3**
- 32. **4**
- 33. **5**
- 34. **6**
- 35. **7**
- 36. **8**
- 37. **9**
- 38. **10**
- 39. **11**
- 40. **12**
- 41. **13**
- 42. **14**
- 43. **15**
- 44. **16**
- 45. **17**
- 46. **18**
- 47. **19**
- 48. **20**
- 49. **21**
- 50. **22**
- 51. **23**
- 52. **24**
- 53. **25**
- 54. **26**

- 55. **27**
- 56. **28**
- 57. **29**
- 58. **30**
- 59. **31**
- 60. **32**
- 61. **33**
- 62. **34**
- 63. **35**
- 64. **36**
- 65. **37**
- 66. **38**
- 67. **39**
- 68. **40**
- 69. **41**
- 70. **42**
- 71. **43**
- 72. **44**
- 73. **45**
- 74. **46**
- 75. **47**
- 76. **48**
- 77. **49**
- 78. **50**
- 79. **51**
- 80. **52**
- 81. **53**
- 82. **54**

- 83. **55**
- 84. **56**
- 85. **57**
- 86. **58**
- 87. **59**
- 88. **60**
- 89. **61**
- 90. **62**
- 91. **63**
- 92. **64**
- 93. **65**
- 94. **66**
- 95. **67**
- 96. **68**
- 97. **69**
- 98. **70**
- 99. **71**
- 100. **72**
- 101. **73**
- 102. **74**
- 103. **75**
- 104. **76**
- 105. **77**
- 106. **78**
- 107. **79**
- 108. **80**
- 109. **81**
- 110. **82**

- 111. **83**
- 112. **84**
- 113. **85**
- 114. **86**
- 115. **87**
- 116. **88**
- 117. **89**
- 118. **90**
- 119. **91**
- 120. **92**
- 121. **93**
- 122. **94**
- 123. **95**
- 124. **96**
- 125. **97**
- 126. **98**
- 127. **99**
- 128. **100**
- 129. **101**
- 130. **102**
- 131. **103**
- 132. **104**
- 133. **105**
- 134. **106**
- 135. **107**
- 136. **108**
- 137. **109**
- 138. **110**

- 139. [111](#)
- 140. [112](#)
- 141. [113](#)
- 142. [114](#)
- 143. [115](#)
- 144. [116](#)
- 145. [117](#)
- 146. [118](#)
- 147. [119](#)
- 148. [120](#)
- 149. [121](#)
- 150. [122](#)
- 151. [123](#)
- 152. [124](#)
- 153. [125](#)
- 154. [126](#)
- 155. [127](#)
- 156. [128](#)
- 157. [129](#)
- 158. [130](#)
- 159. [131](#)
- 160. [132](#)
- 161. [133](#)
- 162. [134](#)
- 163. [135](#)
- 164. [136](#)
- 165. [137](#)
- 166. [138](#)

- 167. **139**
- 168. **140**
- 169. **141**
- 170. **142**
- 171. **143**
- 172. **144**
- 173. **145**
- 174. **146**
- 175. **147**
- 176. **148**
- 177. **149**
- 178. **150**
- 179. **151**
- 180. **152**
- 181. **153**
- 182. **154**
- 183. **155**
- 184. **156**
- 185. **157**
- 186. **158**
- 187. **159**
- 188. **160**
- 189. **161**
- 190. **162**
- 191. **163**
- 192. **164**
- 193. **165**
- 194. **166**

- 195. **167**
- 196. **168**
- 197. **169**
- 198. **170**
- 199. **171**
- 200. **172**
- 201. **173**
- 202. **174**
- 203. **175**
- 204. **176**
- 205. **177**
- 206. **178**
- 207. **179**
- 208. **180**
- 209. **181**
- 210. **182**
- 211. **183**
- 212. **184**
- 213. **185**
- 214. **186**
- 215. **187**
- 216. **188**
- 217. **189**
- 218. **190**
- 219. **191**
- 220. **192**
- 221. **193**
- 222. **194**

223.	195
224.	196
225.	197
226.	198
227.	199
228.	200
229.	201
230.	202
231.	203
232.	204
233.	205
234.	206
235.	207
236.	208
237.	209
238.	210
239.	211
240.	212
241.	213
242.	214
243.	215
244.	216
245.	217
246.	218
247.	219
248.	220
249.	221
250.	222

251. **223**
252. **224**
253. **225**
254. **226**
255. **227**
256. **228**
257. **229**
258. **230**
259. **231**
260. **232**
261. **233**
262. **234**
263. **235**
264. **236**
265. **237**
266. **238**
267. **239**
268. **240**
269. **241**
270. **242**
271. **243**
272. **244**
273. **245**
274. **246**
275. **247**
276. **248**
277. **249**
278. **250**

279. **251**
280. **252**
281. **253**
282. **254**
283. **255**
284. **256**
285. **257**
286. **258**
287. **259**
288. **260**
289. **261**
290. **262**
291. **263**
292. **264**
293. **265**
294. **266**
295. **267**
296. **268**
297. **269**
298. **270**
299. **271**
300. **272**
301. **273**
302. **274**
303. **275**
304. **276**
305. **277**
306. **278**

307. **279**
308. **280**
309. **281**
310. **282**
311. **283**
312. **284**
313. **285**
314. **286**
315. **287**
316. **288**
317. **289**
318. **290**
319. **291**
320. **292**
321. **293**
322. **294**
323. **295**
324. **296**
325. **297**
326. **298**
327. **299**
328. **300**
329. **301**
330. **302**
331. **303**
332. **304**
333. **305**
334. **306**

335. **307**
336. **308**
337. **309**
338. **310**
339. **311**
340. **312**
341. **313**
342. **314**
343. **315**
344. **316**
345. **317**
346. **318**
347. **319**
348. **320**
349. **321**
350. **322**
351. **323**
352. **324**
353. **325**
354. **326**
355. **327**
356. **328**
357. **329**
358. **330**
359. **331**
360. **332**
361. **333**
362. **334**

363. **335**
364. **336**
365. **337**
366. **338**
367. **339**
368. **340**
369. **341**
370. **342**
371. **343**
372. **344**
373. **345**
374. **346**
375. **347**
376. **348**
377. **349**
378. **350**
379. **351**
380. **352**
381. **353**
382. **354**
383. **355**
384. **356**
385. **357**
386. **358**
387. **359**
388. **360**
389. **361**
390. **362**

391. **363**
392. **364**
393. **365**
394. **366**
395. **367**
396. **368**
397. **369**
398. **370**
399. **371**
400. **372**
401. **373**
402. **374**
403. **375**
404. **376**
405. **377**
406. **378**
407. **379**
408. **380**
409. **381**
410. **382**
411. **383**
412. **384**
413. **385**
414. **386**
415. **387**
416. **388**
417. **389**
418. **390**

419. **391**
420. **392**
421. **393**
422. **394**
423. **395**
424. **396**
425. **397**
426. **398**
427. **399**
428. **400**
429. **401**
430. **402**
431. **403**
432. **404**
433. **405**
434. **406**
435. **407**
436. **408**
437. **409**
438. **410**
439. **411**
440. **412**
441. **413**
442. **414**
443. **415**
444. **416**
445. **417**
446. **418**

447. **419**
448. **420**
449. **421**
450. **422**
451. **423**
452. **424**
453. **425**
454. **426**
455. **427**
456. **428**
457. **429**
458. **430**
459. **431**
460. **432**
461. **433**
462. **434**
463. **435**
464. **436**
465. **437**
466. **438**
467. **439**
468. **440**
469. **441**
470. **442**
471. **443**
472. **444**
473. **445**
474. **446**

475. **447**
476. **448**
477. **449**
478. **450**
479. **451**
480. **452**
481. **453**
482. **454**
483. **455**
484. **456**
485. **457**
486. **458**
487. **459**
488. **460**
489. **461**
490. **462**
491. **463**
492. **464**
493. **465**
494. **466**
495. **467**
496. **468**
497. **469**
498. **470**
499. **471**
500. **472**
501. **473**
502. **474**

503. **475**
504. **476**
505. **477**
506. **478**
507. **479**
508. **480**
509. **481**
510. **482**
511. **483**
512. **484**
513. **485**
514. **486**
515. **487**
516. **488**
517. **489**
518. **490**
519. **491**
520. **492**
521. **493**
522. **494**
523. **495**
524. **496**
525. **497**
526. **498**
527. **499**
528. **500**
529. **501**
530. **502**

531. **503**
532. **504**
533. **505**
534. **506**
535. **507**
536. **508**
537. **509**
538. **510**
539. **511**
540. **512**
541. **513**
542. **514**
543. **515**
544. **516**
545. **517**
546. **518**
547. **519**
548. **520**
549. **521**
550. **522**
551. **523**
552. **524**
553. **525**
554. **526**
555. **527**
556. **528**
557. **529**
558. **530**

559. **531**
560. **532**
561. **533**
562. **534**
563. **535**
564. **536**
565. **537**
566. **538**
567. **539**
568. **540**
569. **541**
570. **542**
571. **543**
572. **544**
573. **545**
574. **546**
575. **547**
576. **548**
577. **549**
578. **550**
579. **551**
580. **552**
581. **553**
582. **554**
583. **555**
584. **556**
585. **557**
586. **558**

587. **559**
588. **560**
589. **561**
590. **562**
591. **563**
592. **564**
593. **565**
594. **566**
595. **567**
596. **568**
597. **569**
598. **570**
599. **571**
600. **572**
601. **573**
602. **574**
603. **575**
604. **576**
605. **577**
606. **578**
607. **579**
608. **580**
609. **581**
610. **582**
611. **583**
612. **584**
613. **585**
614. **586**

615. **587**
616. **588**
617. **589**
618. **590**
619. **591**
620. **592**
621. **593**
622. **594**
623. **595**
624. **596**
625. **597**
626. **598**
627. **599**
628. **600**
629. **601**
630. **602**
631. **603**
632. **604**
633. **605**
634. **606**
635. **607**
636. **608**
637. **609**
638. **610**
639. **611**
640. **612**
641. **613**
642. **614**

643. **615**
644. **616**
645. **617**
646. **618**
647. **619**
648. **620**
649. **621**
650. **622**
651. **623**
652. **624**
653. **625**
654. **626**
655. **627**
656. **628**
657. **629**
658. **630**
659. **631**
660. **632**
661. **633**
662. **634**
663. **635**
664. **636**
665. **637**
666. **638**
667. **639**
668. **640**
669. **641**
670. **642**

671. **643**
672. **644**
673. **645**
674. **646**
675. **647**
676. **648**
677. **649**
678. **650**
679. **651**
680. **652**
681. **653**
682. **654**
683. **655**
684. **656**
685. **657**
686. **658**
687. **659**
688. **660**
689. **661**
690. **662**
691. **663**
692. **664**
693. **665**
694. **666**
695. **667**
696. **668**
697. **669**
698. **670**

699. **671**
700. **672**
701. **673**
702. **674**
703. **675**
704. **676**
705. **677**
706. **678**
707. **679**
708. **680**
709. **681**
710. **682**
711. **683**
712. **684**
713. **685**
714. **686**
715. **687**
716. **688**
717. **689**
718. **690**
719. **691**
720. **692**
721. **693**
722. **694**
723. **695**
724. **696**
725. **697**
726. **698**

727. **699**
728. **700**
729. **701**
730. **702**
731. **703**
732. **704**
733. **705**
734. **706**
735. **707**
736. **708**
737. **709**
738. **710**
739. **711**
740. **712**
741. **713**
742. **714**
743. **715**
744. **716**
745. **717**
746. **718**
747. **719**
748. **720**
749. **721**
750. **722**
751. **723**
752. **724**
753. **725**
754. **726**

755. **727**
756. **728**
757. **729**
758. **730**
759. **731**
760. **732**
761. **733**
762. **734**
763. **735**
764. **736**
765. **737**
766. **738**
767. **739**
768. **740**
769. **741**
770. **742**
771. **743**
772. **744**
773. **745**
774. **746**
775. **747**
776. **748**
777. **749**
778. **750**
779. **751**
780. **752**
781. **753**
782. **754**

783. **755**
784. **756**
785. **757**
786. **758**
787. **759**
788. **760**
789. **761**
790. **762**
791. **763**
792. **764**
793. **765**
794. **766**
795. **767**
796. **768**
797. **769**
798. **770**
799. **771**
800. **772**
801. **773**
802. **774**
803. **775**
804. **776**
805. **777**
806. **778**
807. **779**
808. **780**
809. **781**
810. **782**

811. **783**
812. **784**
813. **785**
814. **786**
815. **787**
816. **788**
817. **789**
818. **790**
819. **791**
820. **792**
821. **793**
822. **794**
823. **795**
824. **796**
825. **797**
826. **798**
827. **799**
828. **800**
829. **801**
830. **802**
831. **803**
832. **804**
833. **805**
834. **806**
835. **807**
836. **808**
837. **809**
838. **810**

839. **811**
840. **812**
841. **813**
842. **814**
843. **815**
844. **816**
845. **817**
846. **818**
847. **819**
848. **820**
849. **821**
850. **822**
851. **823**
852. **824**
853. **825**
854. **826**
855. **827**
856. **828**
857. **829**
858. **830**
859. **831**
860. **832**
861. **833**
862. **834**
863. **835**
864. **836**
865. **837**
866. **838**

867. **839**
868. **840**
869. **841**
870. **842**
871. **843**
872. **844**
873. **845**
874. **846**
875. **847**
876. **848**
877. **849**
878. **850**
879. **851**
880. **852**
881. **853**
882. **854**
883. **855**
884. **856**
885. **857**
886. **858**
887. **859**
888. **860**
889. **861**
890. **862**
891. **863**
892. **864**
893. **865**
894. **866**

895. **867**
896. **868**
897. **869**
898. **870**
899. **871**
900. **872**
901. **873**
902. **874**
903. **875**
904. **876**
905. **877**
906. **878**
907. **879**
908. **880**
909. **881**
910. **882**
911. **883**
912. **884**
913. **885**
914. **886**
915. **887**
916. **888**
917. **889**
918. **890**
919. **891**
920. **892**
921. **893**
922. **894**

923. **895**
924. **896**
925. **897**
926. **898**
927. **899**
928. **900**
929. **901**
930. **902**
931. **903**
932. **904**
933. **905**
934. **906**
935. **907**
936. **908**
937. **909**
938. **910**
939. **911**
940. **912**
941. **913**
942. **914**
943. **915**
944. **916**
945. **917**
946. **918**
947. **919**
948. **920**
949. **921**
950. **922**

951. **923**
952. **924**
953. **925**
954. **926**
955. **927**
956. **928**
957. **929**
958. **930**
959. **931**
960. **932**
961. **933**
962. **934**
963. **935**
964. **936**
965. **937**
966. **938**
967. **939**
968. **940**
969. **941**
970. **942**
971. **943**
972. **944**
973. **945**
974. **946**
975. **947**
976. **948**
977. **949**
978. **950**

979. **951**
980. **952**
981. **953**
982. **954**
983. **955**
984. **956**
985. **957**
986. **958**
987. **959**
988. **960**
989. **961**
990. **962**
991. **963**
992. **964**
993. **965**
994. **966**
995. **967**
996. **968**
997. **969**
998. **970**
999. **971**
1000. **972**
1001. **973**
1002. **974**
1003. **975**
1004. **976**
1005. **977**
1006. **978**

1007. **979**
1008. **980**
1009. **981**
1010. **982**
1011. **983**
1012. **984**
1013. **985**
1014. **986**
1015. **987**
1016. **988**
1017. **989**
1018. **990**
1019. **991**
1020. **992**
1021. **993**
1022. **994**
1023. **995**
1024. **996**
1025. **997**
1026. **998**
1027. **999**
1028. **1000**
1029. **1001**
1030. **1002**
1031. **1003**
1032. **1004**
1033. **1005**
1034. **1006**

1035. **1007**
1036. **1008**
1037. **1009**
1038. **1010**
1039. **1011**
1040. **1012**
1041. **1013**
1042. **1014**
1043. **1015**
1044. **1016**
1045. **1017**
1046. **1018**
1047. **1019**
1048. **1020**
1049. **1021**
1050. **1022**
1051. **1023**
1052. **1024**
1053. **1025**
1054. **1026**
1055. **1027**
1056. **1028**
1057. **1029**
1058. **1030**
1059. **1031**
1060. **1032**
1061. **1033**
1062. **1034**

1063. **1035**
1064. **1036**
1065. **1037**
1066. **1038**
1067. **1039**
1068. **1040**
1069. **1041**
1070. **1042**
1071. **1043**
1072. **1044**
1073. **1045**
1074. **1046**
1075. **1047**
1076. **1048**
1077. **1049**
1078. **1050**
1079. **1051**
1080. **1052**
1081. **1053**
1082. **1054**
1083. **1055**
1084. **1056**
1085. **1057**
1086. **1058**
1087. **1059**
1088. **1060**
1089. **1061**
1090. **1062**

1091. **1063**
1092. **1064**
1093. **1065**
1094. **1066**
1095. **1067**
1096. **1068**
1097. **1069**
1098. **1070**
1099. **1071**
1100. **1072**
1101. **1073**
1102. **1074**
1103. **1075**
1104. **1076**
1105. **1077**
1106. **1078**
1107. **1079**
1108. **1080**
1109. **1081**
1110. **1082**
1111. **1083**
1112. **1084**
1113. **1085**
1114. **1086**
1115. **1087**
1116. **1088**
1117. **1089**
1118. **1090**

1119. **1091**
1120. **1092**
1121. **1093**
1122. **1094**
1123. **1095**
1124. **1096**
1125. **1097**
1126. **1098**
1127. **1099**
1128. **1100**
1129. **1101**
1130. **1102**
1131. **1103**
1132. **1104**
1133. **1105**
1134. **1106**
1135. **1107**
1136. **1108**
1137. **1109**
1138. **1110**
1139. **1111**
1140. **1112**
1141. **1113**
1142. **1114**
1143. **1115**
1144. **1116**
1145. **1117**
1146. **1118**

1147. [1119](#)
1148. [1120](#)
1149. [1121](#)
1150. [1122](#)
1151. [1123](#)
1152. [1124](#)
1153. [1125](#)
1154. [1126](#)
1155. [1127](#)
1156. [1128](#)
1157. [1129](#)
1158. [1130](#)
1159. [1131](#)
1160. [1132](#)
1161. [1133](#)
1162. [1134](#)
1163. [1135](#)
1164. [1136](#)
1165. [1137](#)
1166. [1138](#)
1167. [1139](#)
1168. [1140](#)
1169. [1141](#)
1170. [1142](#)
1171. [1143](#)
1172. [1144](#)
1173. [1145](#)
1174. [1146](#)

1175. **1147**
1176. **1148**
1177. **1149**
1178. **1150**
1179. **1151**
1180. **1152**
1181. **1153**
1182. **1154**
1183. **1155**
1184. **1156**
1185. **1157**
1186. **1158**
1187. **1159**
1188. **1160**
1189. **1161**
1190. **1162**
1191. **1163**
1192. **1164**
1193. **1165**
1194. **1166**
1195. **1167**
1196. **1168**
1197. **1169**
1198. **1170**
1199. **1171**
1200. **1172**
1201. **1173**
1202. **1174**

1203. **1175**
1204. **1176**
1205. **1177**
1206. **1178**
1207. **1179**
1208. **1180**
1209. **1181**
1210. **1182**
1211. **1183**
1212. **1184**
1213. **1185**
1214. **1186**
1215. **1187**
1216. **1188**
1217. **1189**
1218. **1190**
1219. **1191**
1220. **1192**
1221. **1193**
1222. **1194**
1223. **1195**
1224. **1196**
1225. **1197**
1226. **1198**
1227. **1199**
1228. **1200**
1229. **1201**
1230. **1202**

1231. **1203**

1232. **1204**

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description

Long description